



National University of Computer and Emerging Sciences, Lahore



Mixture of experts framework for domain adaptive colorization

Mohammad Ibraheem 23L-0771 BS(CS)

Muhammad Zaeem 23L-0886 BS(CS)

Sheikh Muhammad Fareed 23L-0997 BS(CS)

Supervisor: Junaid Hussain Muzamal

Final Year Project

October 4, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: Mixture of experts framework for domain adaptive colorization

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: .....

Name: .....

Signature: .....

Name: .....

Signature: .....

Name: .....

Signature: .....

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

Colorizing grayscale photographs is an underconstrained problem, and a single model must cover very different kinds of content. This project proposes a region-based Mixture-of-Experts framework for domain-adaptive colorization. Grounded-SAM divides a grayscale image into semantic regions, each region is routed by its text prompt to one of four domain experts (portraits and people, nature and landscapes, urban and man-made scenes, animals), and the expert outputs are merged with soft masks. The experts are fine-tuned copies of DDColor, and a new expert can be added without retraining the others. This report presents the method, the system design, and the plan for evaluating it against general-purpose baselines.

Executive Summary

Problem. A grayscale photograph does not contain enough information to determine the original colors of the objects in it, so colorization often has several plausible answers. General-purpose colorization models must handle everything (including people, animals, landscapes, and buildings) with one set of weights, and they are reported to produce desaturated colors, semantic inconsistencies, and color drift between domains.

Proposed solution. This project develops a region-based Mixture-of-Experts framework for domain-adaptive colorization. A pretrained open-vocabulary segmentation model, Grounded-SAM, is used to divide a grayscale photograph into semantic regions using predefined text prompts. Each region is cropped with a surrounding margin (20% by default, adjustable for testing) and routed by its prompt to one of four domain experts: portraits and people, nature and landscapes, urban and man-made scenes, and animals. The experts are fine-tuned copies of the DDColor network with independent weights. They predict only the color channels in the Lab color space, so the brightness and detail of the original photograph are kept unchanged. The outputs are merged with soft mask boundaries to avoid visible seams, and pixels that no region covers are colorized by a fallback expert. A new expert can be added by defining new prompts and training one network. This does not require any modification of the existing pipeline and experts.

Objectives. The project has five objectives:

- identify semantic regions across the four domains;
- train a specialized colorization expert for each domain;
- route detected regions to the correct expert;
- merge the expert outputs with smooth transitions;
- demonstrate the addition of a further expert without retraining the existing ones.

System. The system is a pipeline of subsystems (region decomposition, routing, region extraction and expert colorization, and blending) that a central controller calls in order. A basic web interface lets a user upload a grayscale image and view the result. The system is developed iteratively and is expected to be trained and run on free cloud notebook resources, which limits it to a single GPU of moderate memory.

Evaluation plan. Each stage is measured separately. Segmentation is measured with mean intersection-over-union and routing with routing accuracy. Colorization quality is measured with FID, LPIPS, colorfulness, PSNR, SSIM, and Lab color difference, and seams with the color difference across region

boundaries. The framework is compared with three general-purpose baselines (pretrained DDColor-T, DDColor-T fine-tuned on pooled data from all four domains, and pretrained DDColor-L) on single-domain and mixed-scene test sets. Ablations vary the blending, the crop margin, the routing, and the use of segmentation masks, and the extensibility claim is tested by adding a fifth expert.

Scope and limitations. The project covers still photographs. Video and medical images are outside its scope. The generated colors are plausible interpretations, since historical color accuracy cannot be established from a grayscale image alone. The quality of the result also heavily depends on the quality of outputs produced by the segmentation model(s), because a region that is missed or labeled wrongly is not colorized by the intended expert.

Status and next steps. The literature review, the choice of datasets, the methodology, and the system design are complete, and a standalone prototype of the region decomposition stage has been built. The remaining stages are planned for the second phase: routing and region extraction, fine-tuning of the four experts, blending, the web interface, the evaluation against the baselines, and the extensibility demonstration.

Sustainable Development Goal. The project relates to SDG 11, Target 11.4, the protection of cultural heritage, because colorized versions of historical photographs can supplement the original monochrome records.

Table of Contents

List of Figures **xi**

List of Tables **xii**

1 Introduction **1**

1.1 Purpose of this Document 1

1.2 Intended Audience 2

1.3 Definitions, Acronyms, and Abbreviations 2

1.4 Conclusion 2

2 Project Vision **4**

2.1 Problem Domain Overview 4

2.2 Problem Statement 5

2.3 Problem Elaboration 5

2.4 Goals and Objectives 5

2.5 Project Scope 6

2.6 Sustainable Development Goal (SDG) 7

3 Literature Review / Related Work **8**

3.1 Definitions, Acronyms, and Abbreviations 8

3.1.1 Definitions 8

3.1.2 Acronyms 9

3.2 Detailed Literature Review 9

3.2.1 Foundations of Automatic Image Colorization 10

3.2.2 Transformer, Generative and Diffusion-Based Colorization 12

3.2.3 Mixture-of-Experts and Conditional Computation 14

3.2.4 Semantic Region Decomposition and Routing 16

3.2.5 Datasets and Evaluation Foundations 17

3.3 Literature Review Summary Table 18

3.4 Related Applications and Existing Development Systems 20
3.4.1 Identification of Research and Technological Gap 21
3.5 Conclusion 22

4 Software Requirement Specifications 24
4.1 List of Features 24
4.2 Functional Requirements 25
4.3 Quality Attributes 26
4.3.1 Correctness 26
4.3.2 Modularity 26
4.3.3 Reproducibility 26
4.3.4 Evaluability 27
4.3.5 Extensibility 27
4.3.6 Resource Awareness 27
4.4 Non-Functional Requirements 27
4.4.1 Performance 27
4.4.2 Reusability 27
4.4.3 Maintainability 27
4.4.4 Reliability 28
4.4.5 Usability 28
4.4.6 Compatibility 28
4.4.7 Scalability 28
4.4.8 Concurrent Usage 28
4.5 Assumptions 28
4.6 Use Cases 29
4.7 Hardware and Software Requirements 33
4.7.1 Hardware Requirements 33
4.7.2 Software Requirements 34
4.8 Graphical User Interface 35
4.9 Database Design 36
4.9.1 ER Diagram 36
4.9.2 Data Dictionary 36
4.10 Risk Analysis 37
4.10.1 Semantic Segmentation Risk 37
4.10.2 Routing Risk 37

4.10.3 Dataset and Expert Training Risk . . . 37
4.10.4 Blending Risk . . . 38
4.10.5 Computational Risk . . . 38
4.10.6 Dependency Risk . . . 38
4.10.7 Schedule Risk . . . 38
4.11 Conclusion . . . 39

5 Proposed Approach and Methodology 40
5.1 Overview of the Proposed Framework . . . 40
5.1.1 Pipeline Stages . . . 40
5.1.2 Color Space and Problem Formulation . . . 41
5.1.3 Relation to Existing MoE Approaches . . . 42
5.2 Region Decomposition . . . 42
5.2.1 Segmentation Model . . . 42
5.2.2 Domain Prompt Set . . . 43
5.2.3 Procedure . . . 43
5.2.4 Limitations of the Decomposition Stage . . . 44
5.3 Expert Routing . . . 44
5.3.1 Label-to-Domain Mapping . . . 45
5.3.2 Handling Overlapping Regions . . . 45
5.3.3 Handling Uncovered Pixels . . . 45
5.4 Context-Aware Region Extraction . . . 46
5.4.1 Cropping with Boundary Context . . . 46
5.4.2 Input Resizing and Mask Handling . . . 47
5.4.3 Mapping Expert Outputs Back to the Image . . . 47
5.5 Expert Colorization Models . . . 47
5.5.1 Base Architecture . . . 47
5.5.2 Training Data . . . 48
5.5.3 Training Procedure . . . 49
5.5.4 Inference Procedure . . . 49
5.6 Blending . . . 49
5.6.1 Soft Mask Construction . . . 50
5.6.2 Weighted Chrominance Combination . . . 50
5.6.3 Recombination with the Original Luminance . . . 51
5.7 Extensibility: Adding a New Expert . . . 51

5.7.1 Procedure for Adding an Expert . . . 51
5.7.2 Independence of Existing Experts . . . 52
5.8 Evaluation Protocol . . . 52
5.8.1 Datasets and Splits . . . 52
5.8.2 Region Decomposition Metrics . . . 53
5.8.3 Expert Colorization Metrics . . . 54
5.8.4 Routing Accuracy . . . 54
5.8.5 Blending Metrics . . . 55
5.8.6 Baselines and Ablations . . . 55
5.8.7 Extensibility Test . . . 56

6 High-Level and Low-Level Design 57
6.1 System Overview . . . 57
6.1.1 Purpose and Functionality . . . 57
6.1.2 Basic Design Approach . . . 58
6.1.3 Inputs, Outputs, and Users . . . 58
6.2 Design Considerations . . . 59
6.2.1 Assumptions and Dependencies . . . 59
6.2.2 General Constraints . . . 60
6.2.3 Goals and Guidelines . . . 61
6.2.4 Development Methods . . . 61
6.3 System Architecture . . . 62
6.3.1 Architecture Overview . . . 62
6.3.2 Subsystem Architecture . . . 64
6.3.3 Interaction Scenarios . . . 67
6.3.4 Rationale for the Decomposition . . . 70
6.4 Architectural Strategies . . . 71
6.4.1 Staged Pipeline with a Central Controller . . . 71
6.4.2 Separation of the User Interface from the Pipeline . . . 71
6.4.3 Reuse of Pretrained Models . . . 71
6.4.4 Common Architecture with Independent Weights . . . 72
6.4.5 Registry-Driven Extensibility . . . 72
6.4.6 Chrominance-Only Prediction with Luminance Preservation . . . 72
6.4.7 Memory Management . . . 72
6.4.8 Error Detection and Recovery . . . 73

6.4.9 Control Flow and Concurrency 73
6.5 Domain Model and Class Diagram 73
6.6 Policies and Tactics 74
6.6.1 Tools and Libraries 74
6.6.2 Coding Conventions 75
6.6.3 Testing 75
6.6.4 Reproducibility and Maintenance 75
6.6.5 Traceability 75

List of Figures

2.1 Sustainable Cities and Communities. 7

4.1 Proposed Interface of the Image Colorization System 35

5.1 Overview of the proposed region-based Mixture-of-Experts colorization pipeline. . . 41

6.1 Top-level architecture of the system. 63
6.2 Components of the Region Decomposition subsystem. 65
6.3 Components of the Region Extraction and Expert Colorization subsystem. 66
6.4 Components of the Blending and Reconstruction subsystem. 68
6.5 System Sequence diagram of colorizing an image. 69
6.6 Sequence diagram of adding a new expert. 70
6.7 Class diagram of the run-time pipeline. 74

List of Tables

3.1 Summary of Relevant Literature . . . 18
3.1 Summary of Relevant Literature . . . 19
3.1 Summary of Relevant Literature . . . 20

4.1 Functional Requirements of the Proposed System . . . 25
4.2 Use Case for Colorizing a Grayscale Photograph . . . 29
4.3 Use Case for Adding a New Domain Expert . . . 32
4.4 Data Dictionary for Experimental Artifacts . . . 36

5.1 Initial prompt set for each expert domain. . . . 43
5.2 Mapping from region labels to domain experts. . . . 45
5.3 Training datasets for each expert domain. . . . 48
5.4 Ablations of the proposed framework. . . . 55

6.1 Relation between the stages of the method (Chapter 5) and the subsystems of the design. . . . 58
6.2 Inputs and outputs of the run-time subsystems. . . . 64
6.3 Relation between objectives, components, and evaluation. . . . 75

Chapter 1 Introduction

Image colorization involves assigning plausible colors to grayscale images while preserving their visual structure. It has applications in presenting historical photographs, producing digital media, and supporting visual interpretation. However, grayscale images do not contain enough information to determine the original colors of every object. Different colors may have similar brightness values, making colorization an underconstrained problem with several plausible outputs. Existing research identifies this ambiguity as a source of incorrect semantic colors and limited color richness in automatically colorized images.

Recent approaches have explored how semantic understanding and visual context can improve colorization. DDColor combines a pixel decoder with a query-based color decoder to connect spatial features with semantic color representations and reduce color bleeding. Its findings demonstrate the importance of considering both image structure and object meaning when predicting colors. Nevertheless, these findings do not establish that a single model is inherently incapable of handling multiple domains; rather, they motivate further investigation into alternative ways of organizing colorization expertise.

The proposed project, Mixture of Experts Framework for Domain Adaptive Colorization, investigates a region-based alternative. Instead of assigning an entire photograph to one colorization model, the system will identify semantic regions and route them to experts specializing in portraits and people, nature and landscapes, urban and man-made scenes, and animals. The resulting outputs will be combined into a single color image. The project aims to evaluate whether this division of responsibility improves domain-specific color quality and semantic consistency while maintaining coherence across the complete photograph.

1.1 Purpose of this Document

This document outlines the proposed framework for the FYP Development Phase 1 report, establishing a shared understanding among the development team, supervisor, and evaluation panel by detailing the research motivation, objectives, scope, and planned evaluation while distinguishing completed initial study work from pending implementation and experiments. The central research question investigates whether semantic region decomposition combined with domain-specialized colorization experts can enhance color quality and consistency relative to general-purpose colorization approaches. The methodology integrates pretrained Grounded-SAM for region identification, prompt-based routing mechanisms, separately trained domain-specific experts, and a blending module to synthesize results. Evaluation will comprehensively assess segmentation quality, routing accuracy, domain-level colorization performance, boundary consistency, and overall image quality metrics. While the project aims to produce percep-

tually convincing colorizations, it explicitly acknowledges that historical color authenticity cannot be determined solely from grayscale information.

1.2 Intended Audience

The primary audience for this document comprises the project supervisor and academic evaluation panel, who will assess the project's motivation, feasibility, technical approach, and alignment with its stated objectives, while also supporting the development team by clearly defining the system's boundaries and expected behavior. A secondary audience includes researchers and students interested in image colorization, semantic segmentation, and modular expert-based architectures, as well as practitioners in historical preservation or digital media who may use the document to understand the proposed application and its limitations. Although the document is structured to be accessible, technical knowledge of machine learning and image processing will be helpful for fully interpreting the architecture design and evaluation measures.

1.3 Definitions, Acronyms, and Abbreviations

The following terms are used throughout this document:

FYP: Final Year Project

MoE: Mixture of Experts

SAM: Segment Anything Model

mIoU: Mean Intersection over Union

FID: Frechet Inception Distance

LPIPS: Learned Perceptual Image Patch Similarity

PSNR: Peak Signal-to-Noise Ratio

SSIM: Structural Similarity Index Measure

SDG: Sustainable Development Goal

1.4 Conclusion

In conclusion, the proposed Mixture of Experts Framework for Domain-Adaptive Colorization aims to improve grayscale image colorization by using specialized experts instead of relying on a single general-purpose model. The system will divide an image into meaningful regions and assign them to experts for portraits and people, nature and landscapes, urban and man-made scenes, and animals. By combining semantic region decomposition, expert routing, contextual colorization, and output blending,

the project aims to generate more plausible and visually coherent color images. The framework will also be designed to allow additional domain experts to be incorporated in the future without redesigning the complete system.

Chapter 2 Project Vision

The project envisions a photographic colorization system that combines domain-specific expertise within a single processing framework. A photograph may contain several types of content, and the system will allow different regions to receive different expert treatment. The intended outcome is a working prototype through which the benefits and limitations of this region-based approach can be examined.

2.1 Problem Domain Overview

The project belongs to the domain of automatic image colorization. Its input will be a grayscale photograph, and its output will be a colorized version produced through semantic decomposition, expert routing, specialized processing, and blending. The four initial expert domains are portraits and people, nature and landscapes, urban and man-made scenes, and animals. These domains describe expert responsibilities rather than mutually exclusive categories of complete photographs.

Research on automatic colorization demonstrates the importance of considering multiple plausible colors and the surrounding scene context. Cong et al. [1] proposes generating reference images that guide colorization and permit localized modifications. Their work supports the relevance of contextual color reasoning, although reference generation and interactive editing are not components of the proposed project. The present framework instead investigates whether separately trained domain experts can provide useful specialization.

For region decomposition, the project will use pretrained Grounded-SAM [2] with predefined prompts such as person, animal, building, sky, and vegetation. Grounded-SAM combines Grounding DINO [3] with SAM [4] to support detection and segmentation based on text inputs. This provides a basis for identifying regions associated with the project's expert domains without requiring users to enter prompts manually.

Each identified region will be cropped with an additional boundary whose size will be varied during testing to determine an appropriate value for retaining surrounding context. The cropped region will then be processed by its assigned expert, and soft-mask blending will combine the outputs. In this project, MoE refers to region-level selection among separate colorization networks, rather than token-level routing within a transformer. Together, the related research motivates the use of semantics and context, while the effectiveness of the proposed region–expert combination remains an experimental question.

2.2 Problem Statement

Existing automatic image-colorization methods typically rely on a single model to colorize all types of photographic content. However, research shows that such general-purpose models often produce desaturated colors, semantic inconsistencies, or domain color drift when applied to diverse scenes containing people, animals, landscapes, and urban structures. Developing a unified model that handles these conflicting visual domains remains a recognized challenge. To address this, we are developing a Mixture of Experts framework that routes semantic image regions to domain-specialized colorization experts. The project aims to improve domain-specific color quality and semantic consistency while maintaining coherence across the complete photograph.

2.3 Problem Elaboration

Automatic image colorization must assign plausible colors to diverse visual content despite the absence of explicit chromatic information. Objects with similar grayscale appearances may have different plausible colors, so producing a convincing colorization is not the same as exactly matching a reference image. This ambiguity makes it difficult for a single general-purpose model to handle all types of scenes without encouraging desaturated outputs or semantically inconsistent colors. A photograph may contain people, animals, vegetation, buildings, and other elements simultaneously. The system must identify these regions and associate them with suitable domain-specialized experts. Missed objects, incomplete masks, or ambiguous prompt matches could affect routing and subsequent colorization. Each expert will be trained on whole images from its domain but will receive cropped regions during pipeline operation, with a surrounding boundary intended to retain useful context. Experts operating separately may also make different color decisions for neighboring regions, so soft-mask blending is used to reduce visible seams. The project must assess both local boundary behavior and the quality of the complete output. The framework is further required to support the addition of new experts through extra models and prompt sets without retraining existing ones.

2.4 Goals and Objectives

The overall goal is to develop and evaluate a Mixture of Experts framework that converts grayscale photographs into plausible color images through domain-specialized processing.
The specific objectives are:

- **Identify semantic regions:** Investigate region decomposition across the four selected domains and measure segmentation quality using mIoU on a labeled dataset such as COCO-Stuff [5].

- **Develop domain experts:** Train a specialized colorization model for each domain. Evaluate held-out outputs using FID, LPIPS, colorfulness, and PSNR/SSIM or Lab color difference against reference images.
- **Implement expert routing:** Assign detected regions to appropriate experts using their semantic characteristics and predefined domain prompts. Measure routing accuracy as the fraction of evaluated regions assigned to the correct expert.
- **Integrate expert outputs:** Combine colorized regions while aiming to preserve smooth transitions and overall visual consistency. Evaluate boundary color differences and compare final-image FID and LPIPS with and without the blending module.
- **Demonstrate extensibility:** Add a further expert and its associated prompt set without modifying or retraining the existing experts.

2.5 Project Scope

The project covers the design, development, and evaluation of a working prototype for grayscale photographic colorization. It focuses on four initial domains and supports photographs containing multiple domain types. Its main components are semantic region decomposition, domain-based routing, separately trained colorization experts, contextual cropping, and output blending.
The initial study has identified the following candidate datasets:

- Portraits and people: CelebA, CelebA-HQ, FFHQ, DeepFashion-MultiModal, and COCO person images.
- Nature and landscapes: Places365 and SUN397.
- Urban and man-made scenes: Cityscapes, ADE20K, and urban subsets of Places365.
- Animals: AFHQ, Oxford-IIIT Pets, and animal subsets of COCO.

Experts will be trained using color images from their respective domains, with grayscale inputs derived from the L channel in Lab space. This supplies paired training examples without manually annotating colors. A labeled segmentation dataset such as COCO-Stuff [5] will support evaluation of region decomposition.
The project deliverables will include:

- A working prototype integrating region decomposition, routing, expert inference, and blending.
- Four domain-specialized models trained for the initial expert categories.

- An experimental evaluation covering the objectives and the effect of blending.
- An extensibility demonstration showing incorporation of an additional expert and prompt set.

2.6 Sustainable Development Goal (SDG)

The Sustainable Development Goal targeted by this project is Goal 11: Sustainable Cities and Communities. The project relates to Target 11.4, which focuses on protecting and safeguarding cultural and natural heritage. The proposed colorization framework can support the presentation and interpretation of historical grayscale photographs through supplementary colorized versions. This may make historical visual content more accessible and engaging for viewers and researchers. However, the generated colors will represent plausible interpretations, while the original monochrome photographs will remain the primary historical records. The complete SDG list is shown in Figure 2.1.



Figure 2.1: Sustainable Cities and Communities.
The figure represents SDG 11

Chapter 3 Literature Review / Related Work

This chapter presents the literature review and related technological work relevant to the proposed Mixture-of-Experts framework for domain-adaptive image colorization. The review covers automatic grayscale-to-color image generation, semantic and contextual colorization, generative and transformer-based approaches, Mixture-of-Experts architectures, and semantic image decomposition. Particular attention is given to the limitations of general-purpose colorization models when processing visually heterogeneous domains. The reviewed literature establishes the technical foundations for decomposing an image into semantically meaningful regions and applying specialized colorization models to those regions.

3.1 Definitions, Acronyms, and Abbreviations

3.1.1 Definitions

Image Colorization: The process of adding suitable colors to a black-and-white (grayscale) image to create a realistic color image.

Grayscale Image: A digital image in which each pixel carries only light intensity or brightness information, containing no color data.

Chrominance: The color-related information of an image, commonly represented separately from luminance in color spaces.

Domain: A visually or semantically coherent category of image content with characteristic color distributions and visual structures.

Domain-Specialized Expert: A colorization model trained primarily on images belonging to a particular visual domain.

Mixture of Experts (MoE): An architecture containing multiple specialized models or expert components together with a mechanism that selects or routes inputs to appropriate experts.

Semantic Region: A spatial region of an image corresponding to an identifiable object, material, or scene component.

Routing: The process of assigning an image region to an appropriate specialized expert.

Blending: The process of combining the outputs of multiple experts into a single image while reducing visible discontinuities between neighboring regions.

Color Discontinuity: A measurable difference in predicted color across the boundary between neigh-

boring regions.

Photorealistic Colorization: Colorization intended to produce visually plausible colors and spatially coherent image appearance.

Open-Vocabulary Segmentation: Semantic detection and segmentation that can use textual descriptions or prompts to identify image regions beyond a fixed closed set of categories.

3.1.2 Acronyms

AI: Artificial Intelligence

CNN: Convolutional Neural Network

MoE: Mixture of Experts

GAN: Generative Adversarial Network

ViT: Vision Transformer

FID: Fréchet Inception Distance

LPIPS: Learned Perceptual Image Patch Similarity

PSNR: Peak Signal-to-Noise Ratio

SSIM: Structural Similarity Index

mIoU: Mean Intersection over Union

ΔE: CIE Lab colour difference

Lab: CIE L*a*b* colour space

SAM: Segment Anything Model

RGB: Red, Green, Blue

DINO: DETR with Improved DeNoising Anchor Boxes / Grounding DINO context

3.2 Detailed Literature Review

The **research papers reviewed** in this study are divided into four main categories: **automatic image colorization, semantic and contextual colorization, Mixture-of-Experts architectures, and semantic segmentation**. These reviewed studies provide the technical foundation for analyzing existing approaches and identifying the research gap addressed by the proposed region-routing colorization approach.

3.2.1 Foundations of Automatic Image Colorization

3.2.1.1 Colorful Image Colorization

Zhang, Isola, and Efros introduced a fully automatic deep-learning approach to grayscale image colorization by treating color prediction as a classification problem rather than straightforward deterministic regression [6]. Their approach used a large-scale collection of color images and class rebalancing to address the tendency of colorization models to produce desaturated results. The work demonstrated that the inherent uncertainty of grayscale-to-color mapping should be explicitly considered because multiple plausible colors can correspond to the same luminance information. The authors also introduced a human evaluation in which generated images were compared with ground-truth photographs.

A major strength of this work is that it shows colorization is not a simple task because there can be many possible colors for the same grayscale image. Treating colorization as a prediction problem helped later studies focus on producing more varied and realistic colors. However, it is still a general-purpose model and does not divide the image into different semantic regions or use specialized models for them.

The relationship to the proposed project is important because the proposed system also treats grayscale-to-color mapping as an uncertain prediction task. However, instead of using one general model, it uses multiple specialized experts to handle different image regions. This allows the system to better handle the different types of content found in an image.

3.2.1.2 Let There Be Color!: Joint End-to-End Learning of Global and Local Image Priors

Iizuka, Simo-Serra, and Ishikawa proposed a colorization method that combines global scene information with local image information [7]. The method uses the overall scene to help decide suitable colors, while local information helps produce detailed color predictions.

A key strength of this approach is that it combines scene understanding with local color prediction. This is useful because individual pixels usually do not contain enough information to determine their correct color. However, both types of information are handled within one colorization model rather than through separate domain-specific experts.

The proposed framework extends this idea by first identifying meaningful image regions and then including additional context around them. The approximately 20% (variable to allow for testing) extra region boundary would help preserve surrounding information while allowing specialized experts to process individual regions.

3.2.1.3 Real-Time User-Guided Image Colorization with Learned Deep Priors

Zhang et al. developed a user-guided colorization system where users provide a small number of color hints that are then spread across the image using a CNN-based model [8]. Instead of depending only on automatically predicted colors, the system allows users to give local color information to guide the colorization process. The method also suggests possible colors and uses learned semantic information to apply the user's guidance to relevant areas.

The main strength of this approach is the control it gives to the user. Users can adjust or guide the colors of uncertain regions instead of relying completely on automatic predictions. However, the system depends on user-provided color hints, while the proposed project uses predefined prompts and automatic region decomposition without requiring manual input from the user.

This work is relevant because it shows one way of dealing with uncertainty in image colorization. The proposed framework follows a different approach: instead of relying on the user to handle uncertain regions, it uses semantic routing and domain-specialized experts to deal with differences between image domains.

3.2.1.4 Deep Exemplar-Based Colorization

He et al. proposed a deep exemplar-based method in which a grayscale image is colorized using a separate reference color image [9]. The method includes components that measure the similarity between the reference and target images and then use this information to predict and transfer chrominance. The authors also introduced an automatic reference recommendation method based on semantic and luminance information.

The main strength of this approach is the control provided through reference selection. Different reference images can produce different colorization results, giving users more flexibility compared to a fully automatic method. However, the main limitation compared to the proposed project is that it depends on a reference image. It does not divide the target image into different domain-specific regions or process those regions separately using specialized experts.

The proposed project follows a similar idea that different visual contexts may need different color information. However, instead of using an external reference image, the proposed system gets domain-specific knowledge from separately trained experts.

3.2.2 Transformer, Generative and Diffusion-Based Colorization

3.2.2.1 Colorization Transformer

Kumar, Weissenborn, and Kalchbrenner introduced the Colorization Transformer, which applies self-attention to the colorization problem [10]. The system first generates a low-resolution coarse colorization and subsequently uses parallel networks to obtain higher-resolution output. The transformer architecture is designed to capture long-range dependencies, allowing information from distant image regions to influence color prediction.

The strength of transformer-based colorization is its ability to model relationships extending beyond local convolutional neighborhoods. The approach also supports multiple colorization samples rather than relying exclusively on one deterministic output. A limitation is that the model remains fundamentally general-purpose rather than explicitly specialized according to semantic domains.

The proposed framework can benefit from this literature when designing individual experts. However, the main architectural distinction is that the proposed system places domain specialization outside the individual colorization model by routing semantic regions to separate experts.

3.2.2.2 Towards Vivid and Diverse Image Colorization With Generative Color Prior

Wu et al. investigated the use of a pretrained GAN as a generative color prior [11]. Their approach extracts features representing plausible color distributions and incorporates those features into the colorization process. The resulting method seeks vivid and diverse outputs rather than the desaturated results frequently associated with deterministic prediction.

The approach demonstrates that an explicit generative color prior can improve chromatic richness and diversity. Nevertheless, the color prior is still learned as part of a general colorization framework rather than being explicitly divided according to semantic domains.

The proposed project is related because its objective also includes realistic and perceptually convincing color. The difference is that the proposed project obtains specialization through multiple domain experts rather than through one generalized generative color prior.

3.2.2.3 BigColor

Kim et al. introduced BigColor, a generative color-prior approach intended to improve colorization of complex in-the-wild images [12]. The authors specifically identified limitations in previous generative priors that attempted to model both image structure and color simultaneously. BigColor instead focuses its generative prior on color synthesis while preserving spatial structure.

The principal strength is its focus on separating color generation from structural representation. The method supports diverse colorization results and arbitrary input resolutions. Its limitation in relation to the proposed project is that its specialization is still encoded inside a unified generative architecture rather than explicitly assigned by image region.

The proposed project similarly recognizes that image structure and color prediction are closely related, but takes a different architectural route. Semantic decomposition is performed first, after which each region is assigned to an expert whose training data corresponds to a specific domain.

3.2.2.4 Palette: Image-to-Image Diffusion Models

Saharia et al. introduced Palette, a unified conditional diffusion framework for image-to-image translation, including colorization [13]. The work evaluates diffusion-based translation on several image restoration and transformation tasks and demonstrates the ability of conditional diffusion models to generate plausible image outputs.

Palette is significant because diffusion provides another mechanism for representing the multiple plausible outputs inherent to image-to-image translation. However, the framework is designed as a general image-to-image model and does not implement the proposed region-level domain-specialized expert architecture.

For the proposed work, Palette demonstrates the importance of multimodal generative modeling but does not resolve the specific problem of conflicting color distributions between different semantic domains within the same photograph.

3.2.2.5 DDColor: Towards Photo-Realistic Image Colorization via Dual Decoders

Kang et al. introduced DDColor, an end-to-end colorization architecture containing a pixel decoder and a query-based color decoder [14]. The two components jointly connect multi-scale semantic representations with color queries. The authors specifically address semantic color errors, low color richness, and color bleeding. A colorfulness loss is also used to encourage richer outputs.

DDColor represents a strong general-purpose baseline because it explicitly combines semantic representations with detailed color reconstruction. Its limitation for the proposed project is architectural specialization: it remains a single colorization model rather than a framework where semantically different regions are routed to separate domain experts.

The proposed system can therefore use DDColor-type principles as a conceptual baseline for individual experts, while the principal research contribution remains the region-level MoE architecture. The official implementation also demonstrates that contemporary colorization models can be trained and evaluated

as reusable software components [15].

3.2.2.6 Automatic Controllable Colorization via Imagination

Cong et al. proposed an automatic colorization framework that generates candidate reference images through an imagination module and subsequently refines those references before applying them to colorization [1]. The system addresses the ambiguity of colorization by generating multiple possible visual references and selecting suitable information for the target.

A significant strength is its support for iterative and localized colorization modifications. It demonstrates that automatically generated semantic references can improve colorization flexibility. Its limitation relative to the proposed work is that the generated references are not organized as independently trained domain experts.

The proposal explicitly cites this work as evidence that grayscale-to-color mapping is multimodal. The proposed framework addresses the same ambiguity from another direction: semantic decomposition and domain-specific specialization rather than generated reference-image selection.

3.2.3 Mixture-of-Experts and Conditional Computation

3.2.3.1 Sparsely-Gated Mixture-of-Experts

Shazeer et al. introduced sparsely gated Mixture-of-Experts layers as a mechanism for increasing model capacity while activating only a subset of expert networks for a particular input [16]. A trainable gating network determines which experts participate in processing an input. The work established an important foundation for conditional computation and expert specialization.

The central strength of MoE is the ability to allocate specialized computation to different inputs without requiring every expert to process every input. However, the original application was not image colorization, and its routing mechanism operates at the model/input level rather than according to semantic image regions.

The proposed project adapts the underlying specialization principle to a different problem. Instead of routing complete examples or transformer tokens, the proposed framework routes semantically identified regions to domain-specific colorization experts.

3.2.3.2 Switch Transformers

Fedus, Zoph, and Shazeer developed the Switch Transformer to simplify sparse expert routing and improve the efficiency and stability of large-scale MoE models [17]. The architecture routes each token

to an expert and thereby separates parameter capacity from the amount of computation performed for every token.

Switch Transformers demonstrate that expert routing can be practical when the routing mechanism is carefully designed. Nevertheless, the architecture is primarily concerned with token-level conditional computation and was not designed for semantic image-region colorization.

The proposed project differs by defining experts according to photographic domains and by performing routing at the level of semantically decomposed regions. This distinction is central to the project's intended contribution.

3.2.3.3 Mixture of a Million Experts

He proposed PEER, a parameter-efficient expert retrieval mechanism capable of retrieving from a very large pool of small experts using product-key retrieval [18]. The work investigates how a large number of small experts can provide increased capacity without activating all parameters simultaneously.

The work strengthens the theoretical motivation for modular expert architectures and shows that fine-grained specialization can be scaled efficiently. However, its experiments concern language-modeling tasks rather than image colorization.

The relevance to the proposed project lies primarily in extensibility. The proposal specifically requires the architecture to support incorporation of new domain experts without rebuilding the entire system or modifying existing experts. The concept of modular expert addition is therefore consistent with broader MoE research.

3.2.3.4 Gamma: Toward Generic Image Assessment with Mixture of Assessment Experts

Zhou et al. introduced Gamma, a mixture-of-assessment-experts framework designed to handle diverse image-assessment tasks and datasets [19]. The system uses shared and adaptive experts to capture common and domain-specific information and additionally uses scene-related prompting.

Gamma is especially relevant because it demonstrates that expert specialization can be used to address domain differences within computer vision rather than only in language modeling. Its limitation is that the task is image assessment rather than image generation or colorization.

The proposed project extends the general concept of domain specialization to a generative colorization task. Unlike Gamma, the proposed framework does not use shared/adaptive assessment experts; instead, it trains separate colorization experts for the four domains specified in the proposal.

3.2.3.5 RS-MoEGAN: Sparse Mixture-of-Experts for Cross-Modal Remote Sensing Image Generation

Fan et al. proposed RS-MoEGAN, which applies sparse Mixture-of-Experts routing to cross-modal remote-sensing image generation [20]. The motivation is particularly relevant to the proposed project because different land-cover types can impose conflicting requirements on image generation. RS-MoEGAN uses a learned router to assign image tokens to specialized experts.

The work provides direct evidence that MoE can be useful when a generative vision problem contains conflicting domain-specific requirements. Its main limitation relative to the proposed project is the routing granularity and application domain: it uses token-level routing in remote sensing rather than semantic region-level routing for photographic colorization.

The proposed project is closely related conceptually. Both approaches seek to avoid forcing a single generative model to learn conflicting visual mappings. The proposed project differs by decomposing photographic images into semantic regions and routing those regions to four explicitly defined colorization domains.

3.2.4 Semantic Region Decomposition and Routing

3.2.4.1 Grounded-SAM

Ren et al. introduced Grounded-SAM, an assembly of Grounding DINO and the Segment Anything Model that enables open-vocabulary detection and promptable segmentation [2]. The system can identify and segment regions according to textual prompts, providing a flexible mechanism for extracting semantically meaningful regions from images.

This capability is particularly suitable for the proposed architecture. The project proposal specifies predefined textual prompts such as person, animal, building, sky, and vegetation and uses the resulting masks to determine which expert should process each region.

A limitation is that Grounded-SAM itself is not a colorization system. Its purpose in the proposed architecture is therefore infrastructural: it supplies the semantic decomposition required by the routing mechanism. The proposed project investigates whether this decomposition can serve as the basis for domain-specialized colorization.

3.2.4.2 Instance-Aware Image Colorization

Su et al. (2020) proposed an instance-aware method for colorizing images containing multiple objects[21]. A pretrained Mask R-CNN detects objects and provides bounding-box crops. One instance colorization

network processes all detected crops, while a separate network processes the full image. The networks have the same architecture but different weights. A learned fusion module combines their intermediate features to predict the final colors.

The authors reported improved PSNR, SSIM, and LPIPS results over the compared methods on ImageNet, COCO-Stuff, and Places205. They also found that directly blending separate object and full-image color predictions produced inconsistencies. This finding highlights the need to evaluate color consistency when combining independently processed regions.

This work is closely related to the proposed project because both use regional processing to produce a complete color image. However, Su et al. use one instance colorization network across object types rather than separate domain experts. The proposed system will use Grounded-SAM masks to identify regions and route them to DDColor-based experts fine-tuned for people, nature, urban scenes, and animals. Their colorized outputs will be combined using soft masks. The project therefore investigates whether domain specialization improves colorization and whether new experts can be added without retraining existing ones. Object-level colorization itself is treated as established prior work.

3.2.5 Datasets and Evaluation Foundations

The proposal identifies several datasets for constructing domain-specific training sets, including CelebA, CelebA-HQ, FFHQ, DeepFashion-MultiModal, COCO, Places365, SUN397, Cityscapes, ADE20K, AFHQ, and Oxford-IIIT Pets. The proposal also specifies that grayscale training inputs are derived from the L channel of Lab-space color images and that a labeled segmentation dataset such as COCO-Stuff [5] can be used to evaluate region decomposition.

COCO provides object-level annotations and instance segmentations in complex everyday scenes [22], and COCO-Stuff extends these images with pixel-level labels for background classes such as sky, grass, tree, and water [5]. Because it covers both object and background classes, COCO-Stuff is the dataset used to evaluate region decomposition and routing in this project. ADE20K contains dense scene and object annotations and provides a scene-parsing benchmark covering 150 object and stuff categories [23], and Cityscapes provides detailed semantic and instance annotations for urban street scenes [24]. Both are relevant to the urban and man-made domain and serve as sources of training images for its expert.

The 2025 survey by Anwar et al. provides an extensive taxonomy of colorization approaches, datasets, metrics, and evaluation protocols [25]. It is particularly useful for the proposed project because it confirms the importance of evaluating colorization through multiple complementary measures rather than relying on a single image-quality metric.

3.3 Literature Review Summary Table

Table 3.1: Summary of Relevant Literature

Author	Method	Results	Limitations
Zhang et al. [6]	Uses a CNN to predict color classes and gives more importance to rare colors.	Produced brighter colors and achieved a 32% human fooling rate.	Uses one model for all image content and may assign wrong colors to unclear objects.
Larsson et al. [26]	Predicts pixel color distributions using basic and semantic image features.	Performed better than earlier automatic colorization methods.	Does not use separate models for different semantic regions.
Iizuka et al. [7]	Combines overall scene information with local image features in one CNN.	Produced realistic results across several types of images.	Processes all regions using the same network.
Zhang et al. [8]	Uses a CNN and color hints provided by the user.	Produced realistic results in real time with limited user input.	Requires manual guidance for difficult regions.
He et al. [9]	Transfers colors from a reference image using learned similarity and color propagation.	Produced customizable results, even with less related reference images.	Requires a suitable reference image and an additional selection process.
Wu et al. [11]	Uses color information learned by a pretrained GAN to guide colorization.	Produced vivid and varied colors in a single forward pass.	Uses one general color prior instead of separate domain experts.
Kim et al. [12]	Uses a BigGAN inspired network to learn color information while keeping image structure.	Produced different plausible results and supported different image sizes.	Does not send semantic regions to separate experts.
Saharia et al. [13]	Uses a conditional diffusion model for colorization and other image tasks.	Performed better than several GAN and regression methods.	It is a general image translation model without region based experts.

Table 3.1: Summary of Relevant Literature

Author	Method	Results	Limitations
Kumar et al. [10]	Uses a transformer for initial color prediction followed by networks that increase image resolution.	Produced detailed and varied colorizations with strong FID and human evaluation results.	Uses one transformer model instead of separate domain experts.
Kang et al. [14]	Uses a pixel decoder and a color decoder to connect semantic features with color predictions.	Improved color quality and reduced color bleeding.	Remains a single general purpose colorization model.
Cong et al. [1]	Generates possible reference images and improves them before colorization.	Allowed local changes and more control over the final colors.	Depends on generated references instead of trained domain experts.
Shazeer et al. [16]	Uses a routing network to activate only selected experts for each input.	Increased model capacity without using every expert at the same time.	Was designed mainly for language tasks, not image colorization.
Fedus et al. [17]	Routes each token to one expert using a simpler sparse routing method.	Improved training speed while supporting very large models.	Uses token routing for language models rather than image region routing.
He [18]	Uses product key retrieval to select experts from a very large expert pool.	Improved the balance between performance and computation in language models.	Does not directly address image colorization or visual region routing.
Zhou et al. [19]	Uses shared and specialized experts for different image assessment datasets.	Improved assessment across several image types and datasets.	Focuses on image assessment rather than image generation or colorization.
RS-MoEGAN [20]	Uses a router to send image tokens to specialized experts in a Vision Transformer.	Helped handle different visual regions while using only selected experts.	Focuses on remote sensing images rather than normal photograph colorization.

Table 3.1: Summary of Relevant Literature

Author	Method	Results	Limitations
Grounded-SAM [2]	Combines text based object detection with the Segment Anything Model.	Produces masks for objects identified through text prompts.	Performs segmentation only and does not colorize images.
Zhou et al. [23]	Provides ADE20K images with detailed labels for 150 classes.	Supports the training and evaluation of scene segmentation models.	It is a general segmentation dataset, not a colorization dataset.
Lin et al. [22]	Provides COCO images with object and instance segmentation labels.	Contains 328,000 images and about 2.5 million labeled objects.	Was created for object recognition rather than image colorization.
Cordts et al. [24]	Provides semantic and instance labels for urban street images.	Contains 5,000 detailed and 20,000 coarse annotations.	Mainly covers urban streets and has limited nonurban content.
Anwar et al. [25]	Reviews colorization methods, datasets, training methods, and evaluation measures.	Provides a useful overview and comparison of existing colorization research.	Does not propose a region based Mixture of Experts colorization system.

The literature demonstrates three recurring characteristics of modern colorization research. First, grayscale-to-color prediction is inherently ambiguous, motivating probabilistic, generative, or multimodal approaches. Second, semantic and contextual information is essential for preventing incorrect or visually inconsistent colors. Third, MoE research provides a mechanism for separating conflicting knowledge across specialized components.

3.4 Related Applications and Existing Development Systems

3.4.0.1 DDColor

DDColor is an open source implementation of the ICCV 2023 dual decoder colorization architecture [14, 15]. The project provides pretrained models, training scripts, inference functionality, and model export support.

Its principal strength is the availability of a complete research implementation and a colorization architecture explicitly designed to reduce semantic color errors and color bleeding. Its limitation relative to

the proposed project is that the colorization model remains general purpose rather than consisting of separate experts for people, nature, urban and man made scenes, and animals.

3.4.0.2 Colorful Image Colorization

The Colorful Image Colorization project provides an implementation of the ECCV 2016 colorization method and supports automatic grayscale to color conversion [6, 27]. Its major strength is its simple automatic inference pipeline and established research foundation. Its principal limitation relative to the proposed project is that it applies a general colorization model rather than selecting domain specific experts for different semantic regions.

3.4.0.3 DeOldify

DeOldify is an open source image and film colorization project intended for the restoration and colorization of historical images and footage. Its documentation describes both image and video colorization and emphasizes improvements in artifacts, detail, and color rendering.

Its principal strength is its practical restoration capability and support for both image and video scenarios. However, video is explicitly outside the current scope of the proposed project, and DeOldify does not implement the proposed four domain semantic routing architecture. The project therefore serves as a practical general purpose comparison rather than an architectural equivalent.

3.4.0.4 DeepAI Image Colorizer

DeepAI provides an accessible image colorization application through which users can upload images and request automatic colorization.

The main advantage of DeepAI Image Colorizer is its accessibility through an application interface, which removes the need for users to train or operate a research model. However, its underlying model architecture and routing strategy are not exposed sufficiently to establish the type of domain specialized processing proposed in this project. Consequently, it is useful as an application level comparison rather than as an architectural baseline.

3.4.1 Identification of Research and Technological Gap

The reviewed literature establishes substantial progress in automatic image colorization. CNN based methods demonstrated practical automatic color prediction, transformer approaches improved long range reasoning, generative priors improved color diversity, diffusion models introduced powerful multimodal image generation, and recent systems such as DDColor improved semantic and perceptual quality.

Nevertheless, the dominant colorization formulation remains based on a single general purpose model. The proposal identifies the underlying issue as the need for one model to approximate color distributions across substantially different visual domains.

Mixture of Experts research provides evidence that conflicting knowledge can be separated among specialized experts. RS-MoEGAN is particularly relevant because it uses expert routing for different visual generation requirements [20], while Gamma demonstrates domain aware experts in image assessment [19].

However, the reviewed literature does not establish a directly equivalent system that combines the following components:

1. Semantic region decomposition of a photographic image.
2. Assignment of each region to one of four domain specific colorization experts.
3. Approximately 20% contextual boundary expansion around each selected region.
4. Independent expert colorization.
5. Soft mask blending of all expert outputs into one image.
6. Addition of new experts through new prompts without modifying or retraining existing experts.

This combination represents the technological gap addressed by the proposed development. The project therefore does not attempt to introduce colorization itself as a new problem. Instead, it investigates whether **domain specialization through semantic region level Mixture of Experts routing** can improve the handling of heterogeneous photographs compared with a monolithic colorization approach.

3.5 Conclusion

The literature review shows that image colorization has developed from traditional and CNN based methods to transformer, GAN, diffusion, exemplar based, and controllable generative approaches. These developments have improved color richness, semantic consistency, visual quality, and diversity. However, grayscale to color mapping remains uncertain, especially when a single photograph contains several visually different domains.

The reviewed Mixture of Experts research provides another useful approach in which different or conflicting types of knowledge can be handled by specialized experts and selected through a routing mechanism. Semantic segmentation systems such as Grounded-SAM provide a way to identify different image regions according to their visual and semantic characteristics.

The proposed project combines these ideas into a framework for photographic image colorization. It focuses on four domains: people, nature and landscapes, urban and man made scenes, and animals. The framework combines semantic decomposition, domain routing, specialized colorization, contextual region expansion, and soft mask blending. The evaluation framework measures segmentation, routing, individual expert quality, boundary consistency, and final image quality. The final system is intended to be a working prototype that can also support the addition of new experts without requiring the existing architecture to be redesigned.

Chapter 4 Software Requirement Specifications

This project is a research prototype for testing domain adaptive image colorization with a Mixture of Experts framework. It is different from a normal software application because it does not need user accounts, transactions, or support for many users at the same time. Its main tasks are model training, semantic region detection, expert selection, colorization, blending, and evaluation.

The system also keeps the output of each stage for inspection and debugging purposes. This is necessary because a good final image does not always mean that the segmentation and routing decisions were correct. For this reason, segmentation, routing, expert outputs, and blending will be checked separately before evaluating the complete result.

This chapter explains what the system must do and the conditions under which it will be developed. It covers the main features, functional and nonfunctional requirements, quality attributes, assumptions, hardware and software needs, use cases, data handling, and possible risks.

4.1 List of Features

The main features planned for the prototype are listed below:

- Loading grayscale photographs as input.
- Finding semantic regions with Grounded SAM and predefined text prompts.
- Dividing detected regions into four domains: people, nature and landscapes, urban and man made scenes, and animals.
- Extracting each region with approximately 20% extra surrounding area for context.
- Sending each region to the expert that matches its domain.
- Colorizing different regions separately through their assigned experts.
- Using more than one expert when a photograph contains multiple domains.
- Joining the colorized regions through soft mask blending.
- Displaying intermediate outputs such as masks, routing decisions, expert results, and blended images.
- Measuring performance with segmentation mIoU, routing accuracy, FID, LPIPS, colorfulness, PSNR, SSIM, and Lab color difference where needed.
- Adding new experts and prompt sets without retraining the existing experts.

Together, these features make it possible to build and test region based expert routing for photographic image colorization.

4.2 Functional Requirements

The following requirements describe the main operations of the system. Each one includes a simple result that can be used to confirm that it works.

Table 4.1: Functional Requirements of the Proposed System

ID	Requirement	Acceptance Evidence
FR1	The system shall take a grayscale photograph as input.	A valid grayscale image is loaded without an error.
FR2	The system shall find semantic regions using predefined prompts and Grounded SAM.	Masks are produced for the supported regions in the image.
FR3	The system shall extract every selected region with approximately 20% extra surrounding area.	The crop contains the detected region and some nearby image content.
FR4	The system shall assign each region to a suitable domain expert.	An expert assignment is produced for every selected region.
FR5	The system shall colorize each region through its assigned expert.	Every routed region produces a colorized result.
FR6	The system shall allow different experts to process the same photograph.	An image with multiple domains can produce results from more than one expert.
FR7	The system shall combine the expert outputs using soft mask blending.	The separate expert results are joined into one color image.
FR8	The system shall keep every region in its original position during reconstruction.	Each colorized region appears in the correct place in the final image.
FR9	The system shall display each detected region, its assigned expert, and its colorized output during the current session.	The interface shows the masks, routing decisions, and expert outputs without permanently saving them.
FR10	The system shall evaluate segmentation, routing, expert colorization, and the final image.	The required measurements are calculated and displayed for the evaluated stages.

ID	Requirement	Acceptance Evidence
FR11	The system shall allow a new expert and prompt set to be added without retraining existing experts.	The new expert works while the existing experts remain unchanged.
FR12	The system shall send a region to the fallback expert when it cannot be assigned to a suitable domain-specific expert.	A region with no suitable domain-specific assignment is processed by the fallback expert.

The link between a region, its expert, its output, and its location in the photograph must remain clear throughout the pipeline. Otherwise, a wrong routing decision could be hidden after the regions are blended together.

4.3 Quality Attributes

The most important quality goals for this system are correctness, modularity, reproducibility, evaluability, extensibility, and sensible use of hardware resources.

4.3.1 Correctness

Detected regions must stay in the correct position and size when the final image is rebuilt. The system must also keep each region linked to the expert that processed it. No region should be moved, removed, or resized incorrectly during reconstruction.

4.3.2 Modularity

Segmentation, region extraction, routing, colorization, blending, and evaluation should be kept as separate modules. This will make it easier to test or replace one part without changing the rest of the system.

4.3.3 Reproducibility

The project should keep a record of training settings, dataset splits, expert models, prompts, random seeds, and evaluation settings. This will allow the same experiment to be run again under similar conditions.

4.3.4 Evaluability

The system must keep semantic masks, expert assignments, individual expert results, and blended outputs available for inspection. These outputs are needed to judge each stage instead of judging everything from the final image alone.

4.3.5 Extensibility

It should be possible to add a new domain expert and its prompts without retraining the existing models. The routing configuration should clearly show which prompts belong to each expert.

4.3.6 Resource Awareness

The implementation must work within the available GPU memory, RAM, storage, and training time. Early experiments will therefore use smaller images, batches, and datasets before moving to larger settings.

4.4 Non-Functional Requirements

These requirements describe how the prototype should behave during development and experimentation.

4.4.1 Performance

Segmentation, routing, colorization, and blending should finish within a practical amount of time on the available hardware. A strict real time target is not needed because the system is being built for experiments, not live public use.

Image resolution and batch size should be adjustable so that training can be matched to the available computing power.

4.4.2 Reusability

The main components should be written as reusable modules. Shared image processing and evaluation functions should be used by all experts instead of being copied into several files.

4.4.3 Maintainability

Model paths, prompts, dataset locations, and experiment settings should be kept in configuration files rather than hard coded throughout the program. The source code should also be divided into clear and manageable modules.

4.4.4 Reliability

Failures during segmentation, routing, model loading, or colorization should be recorded. The system should not present an incomplete result as successful when one of its main stages has failed.

4.4.5 Usability

The prototype may be run through scripts, configuration files, notebooks, or a basic interface. The steps for selecting an image, loading the models, running the pipeline, and finding the results should be easy to follow.

4.4.6 Compatibility

The system should accept common image formats such as PNG and JPEG. The required versions of Python, the deep learning framework, CUDA, and other libraries should be documented.

4.4.7 Scalability

The first version will contain four experts, but the structure should allow more experts to be added later. Supporting many users at the same time is not part of this project.

4.4.8 Concurrent Usage

The prototype is intended for use by the project team during experiments. For this reason, there is no fixed target for concurrent users, request throughput, or continuous service availability.

4.5 Assumptions

The project is based on the following assumptions:

- Suitable image datasets are available for training the first four experts.
- The datasets contain enough variety for people, nature and landscapes, urban and man made scenes, and animals.
- Grounded SAM and its pretrained models are available for the experiments.
- Predefined text prompts can identify the main concepts needed by the four domain setup.
- Training images can be converted to grayscale by using the L channel of the Lab color space.
- The original color images are available as references during training and evaluation.

- A labeled dataset such as ADE20K or COCO is available for testing segmentation.
- Ground truth color images are available for measurements that need reference images.
- The first version focuses only on photographic image colorization.
- Video and medical image colorization are outside the current scope.
- The initial implementation uses approximately 20% extra context around each region.
- The amount of extra context may be tested later if enough time and computing resources are available.

These assumptions set the starting conditions for developing and testing the prototype.

4.6 Use Cases

This section presents the main computational scenarios required by the proposed research system. Since the project does not define conventional application users, the actor used in the use cases is the **Project Operator or Experimenter**. This actor represents the person executing the prototype, configuring experiments, and examining the generated results.

Table 4.2: Use Case for Colorizing a Grayscale Photograph

Name	Colorize Grayscale Photograph
Actors	Project Operator or Experimenter
Summary	The operator provides a grayscale photograph. The system identifies semantic regions, routes the regions to suitable experts, colorizes the regions, combines the expert outputs, and produces a final color image.
Pre-Conditions	The colorization pipeline, Grounded SAM, and the supported domain experts must be available. A valid grayscale photograph must be provided.
Post-Conditions	A final colorized image and the corresponding intermediate outputs are produced.
Special Requirements	The system must use predefined prompts and should not require manual region selection. Sufficient memory must be available to load the segmentation and expert models.
Basic Flow	
Actor Action	**System Response**

1	The operator provides a grayscale photograph.	1	The system validates and loads the input image.
2	The operator starts the colorization process.	2	The system passes the image to the semantic region decomposition component.
3	The operator waits while semantic decomposition is performed.	3	Grounded SAM generates semantic region masks using the predefined prompts.
4	The operator waits while the detected regions are prepared.	4	The system extracts each selected region with approximately 20% additional surrounding context.
5	The operator waits while routing is performed.	5	The routing component assigns each region to the appropriate domain expert.
6	The operator waits while the experts process their assigned regions.	6	Each selected expert produces a colorized version of its assigned region.
7	The operator waits while reconstruction is completed.	7	The blending component combines the expert outputs using soft region boundaries.
8	The operator examines the generated result.	8	The system provides the final color image and its intermediate outputs for evaluation.
Alternative Flow			
1A	The operator provides an invalid or unsupported image.	1A	The system rejects the input and displays or records an appropriate error message.
3A	No supported semantic region is identified.	3A	The system sends the whole image to the fallback expert and records that no domain region was found.
5A	A region cannot be associated with a supported expert.	5A	The system sends the region to the fallback expert and records the routing limitation.

6A	An expert model fails to process its assigned region.	6A	The system records the failure and does not present the incomplete result as a successful final output.

Table 4.3: Use Case for Adding a New Domain Expert

Name	Add New Domain Expert		
Actors	Project Operator or Experimenter		
Summary	The operator adds a new domain expert and its associated prompt set without redesigning the framework or retraining the existing experts.		
Pre-Conditions	A trained expert model, its configuration, and its corresponding prompt set must be available.		
Post-Conditions	The new expert becomes available to the routing framework while the existing experts remain unchanged.		
Special Requirements	Existing expert models must not be retrained or modified when the new expert is added.		
Basic Flow			
Actor Action		**System Response**	
1	The operator prepares a trained expert model and its configuration.	1	The system receives and validates the new expert configuration.
2	The operator defines the corresponding textual prompt set.	2	The routing configuration associates the new prompts with the new expert.
3	The operator starts the expert registration process.	3	The system registers the new expert and its prompt mapping.
4	The operator verifies the existing expert configuration.	4	The system keeps all existing experts available without retraining or modification.
5	The operator performs a routing test using an image from the new domain.	5	The system routes supported regions from the new domain to the newly added expert.
Alternative Flow			
2A	The operator provides an incomplete or invalid prompt configuration.	2A	The system rejects the configuration and records the validation problem.
3A	The new expert is incompatible with the required input or output format.	3A	The system does not register the expert and reports the compatibility issue.

5A	The prompt set does not correctly identify the intended domain.	5A	The system records or flags the configuration problem instead of routing regions incorrectly.

These use cases represent the main operational behavior of the prototype. The first use case covers the complete colorization pipeline, while the second demonstrates the extensibility requirement of the proposed architecture.

4.7 Hardware and Software Requirements

This section describes the hardware and software environment required to develop, train, execute, and evaluate the proposed system.

4.7.1 Hardware Requirements

The project requires access to hardware capable of executing modern deep learning models. GPU acceleration is expected to be important for training the colorization experts and running the segmentation and colorization pipeline efficiently.

The required GPU memory depends on the selected expert architecture, image resolution, and batch size. Exact memory requirements will therefore be determined through initial training experiments rather than assuming a fixed value before implementation.

Adequate system memory and storage are required for maintaining training datasets, model checkpoints, generated outputs, semantic masks, and evaluation results. Smaller training batches or image resolutions may be used if hardware limitations are encountered.

The recommended hardware includes:

- A CUDA compatible GPU for training and inference.
- At least 8 GB of system memory for initial development.
- Sufficient storage for datasets, model checkpoints, and experimental outputs.
- A multicore processor for dataset preparation and image processing.

Cloud based GPU resources may be used when the available local hardware is insufficient for training or large scale evaluation.

4.7.2 Software Requirements

The project will use Next.js for the frontend and FastAPI for the backend. The model training, image processing, routing, blending, and evaluation components will be implemented in Python. The required software components are:

- Next.js for developing the web-based user interface of the system.
- HTML and CSS for designing and styling the application interface.
- Node.js and npm for running the Next.js development environment and managing frontend packages.
- FastAPI for developing the backend APIs and connecting the frontend with the image colorization pipeline.
- Python for implementing semantic segmentation, expert routing, image colorization, output blending, and evaluation.
- PyTorch for training and running the domain-specific colorization experts.
- Grounded SAM and its pretrained components for detecting and segmenting semantic regions within an image.
- OpenCV and Pillow for image conversion, cropping, resizing, mask processing, boundary expansion, and output blending.
- NumPy and related scientific computing libraries for numerical and image-processing operations.
- Evaluation libraries for calculating mIoU, FID, LPIPS, colorfulness, PSNR, SSIM, and Lab color difference.
- Jupyter Notebook or Google Colab for model training, testing, and recording experimental results.
- Git and GitHub for source code management and version control.
- Visual Studio Code as the main development environment.
- CUDA and compatible GPU drivers for faster model training and inference.

The Next.js frontend will allow the user to upload a grayscale image and view the semantic masks, assigned experts, individual expert outputs, and final colorized image. The FastAPI backend will receive the image, run the Python-based colorization pipeline, and return the generated results to the frontend. Model configurations, prompts, dataset splits, and experiment settings will also be stored to support the reproduction of important experiments.

4.8 Graphical User Interface

The proposed project is primarily a research prototype and does not require a complex graphical user interface. The system is operated through a basic web interface built with Next.js, which communicates with the Python pipeline through a FastAPI backend (Section 6.3). The pipeline can also be run from scripts and notebooks without the interface, which the evaluation experiments use.

The web interface provides the following functions:

- Selection or upload of a grayscale photograph.
- Initiation of the colorization pipeline.
- Display of the generated semantic masks.
- Display of the expert assigned to each region.
- Display of individual expert outputs.
- Display and saving of the final blended color image.
- Display or export of the calculated evaluation results.

Figure 4.1 shows the proposed interface of the system.

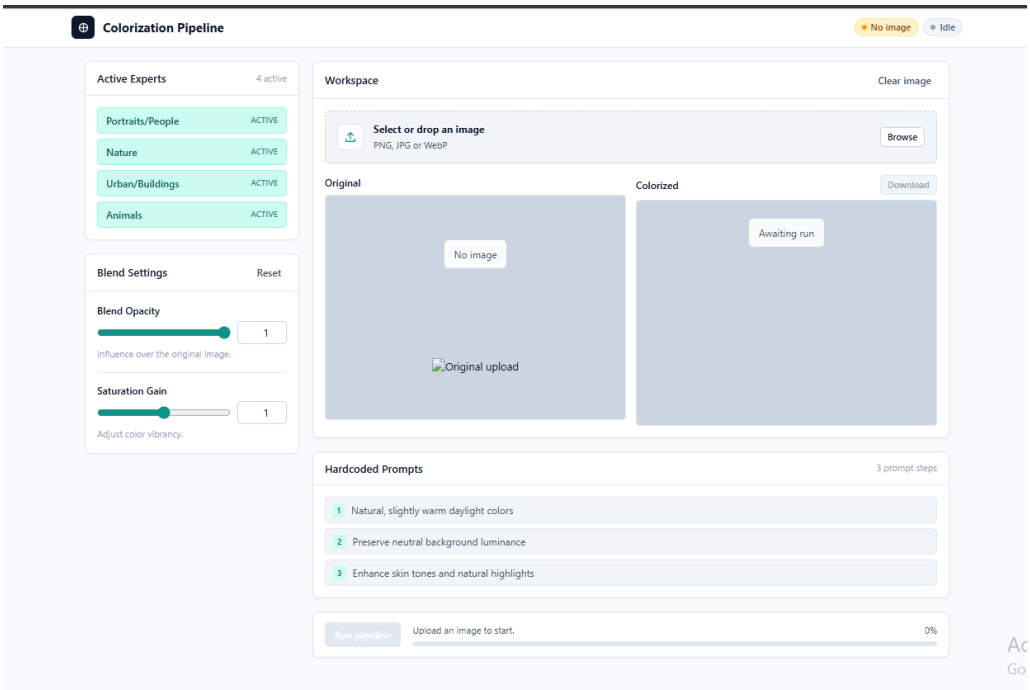


Figure 4.1: Proposed Interface of the Image Colorization System

The interface displays the intermediate results because each stage of the system must be evaluated separately. The final colorized image alone is not enough to verify the segmentation and routing results.

4.9 Database Design

The proposed system does not require a conventional relational or NoSQL database. It does not maintain user accounts, transactions, or continuously changing application records. Experimental information can be maintained using directories, configuration files, CSV files, JSON files, and model checkpoint files.

4.9.1 ER Diagram

An ER diagram is not required because the proposed prototype does not use a relational database. The system stores experimental artifacts as files rather than as persistent relational entities.

The logical relationship between the principal experimental artifacts is that one input image may produce multiple semantic regions, each semantic region is assigned to one expert, each expert produces a colorized region, and the colorized regions are combined into one final image.

4.9.2 Data Dictionary

Table 4.4 describes the main data artifacts maintained by the system.

Table 4.4: Data Dictionary for Experimental Artifacts

Data Item	Format	Description
Input Image	PNG or JPEG	The grayscale photograph provided to the colorization pipeline.
Ground Truth Image	PNG or JPEG	The original color image used as a reference during training or evaluation.
Semantic Mask	PNG, tensor, or array	A mask identifying the pixels associated with a detected semantic region.
Region Label	Text	The semantic concept detected using a predefined textual prompt.
Expert Identifier	Text or integer	The identifier of the expert assigned to process a semantic region.
Contextual Crop	Image or tensor	The selected semantic region together with approximately 20% additional surrounding context.
Expert Output	Image or tensor	The colorized result generated by an individual expert.

Data Item	Format	Description
Final Output	PNG or JPEG	The complete color image generated after soft mask blending.
Prompt Configuration	JSON, YAML, or text	The predefined prompts and their associations with the available experts.
Model Configuration	JSON or YAML	The architecture, training, and inference settings of an expert model.
Model Checkpoint	Model file	The learned parameters of a trained colorization expert.
Evaluation Record	CSV or JSON	The segmentation, routing, expert, blending, and final image evaluation results.

4.10 Risk Analysis

The proposed system contains several connected components. A failure in an earlier component may affect the later stages of the pipeline. The main technical and project risks are described below.

4.10.1 Semantic Segmentation Risk

Incorrect semantic segmentation may cause image regions to be missed or incorrectly identified. Since routing depends on the generated semantic masks, segmentation errors can affect all remaining stages of the pipeline.

This risk will be monitored using segmentation mIoU and by visually inspecting representative segmentation results.

4.10.2 Routing Risk

Some regions may contain visual characteristics associated with more than one domain. This can make expert selection difficult. Incorrect routing may cause a region to be processed by an unsuitable expert.

Routing accuracy will therefore be evaluated independently using labeled test regions. Uncertain or unsupported regions should also be recorded instead of being assigned silently to an unsuitable expert.

4.10.3 Dataset and Expert Training Risk

The datasets available for the different domains may vary in size, quality, balance, and diversity. This may cause some experts to perform better than others and may make direct comparison difficult.

Domain specific datasets will be prepared separately, and each expert will be evaluated using held out images from its own domain.

4.10.4 Blending Risk

Outputs generated by different experts may contain noticeable color differences near the region boundaries. These differences may produce visible seams when the independently colorized regions are combined.

Contextual region expansion and soft mask blending will be used to reduce this problem. The effectiveness of blending will also be evaluated by comparing results produced with and without the blending component.

4.10.5 Computational Risk

Training multiple experts and running segmentation models may require significant computational resources. Hardware limitations may restrict the training resolution, batch size, dataset size, or number of experiments that can be completed.

Initial experiments will use manageable dataset sizes and training configurations. Larger experiments will only be performed after the resource requirements have been measured.

4.10.6 Dependency Risk

The project depends on external libraries, pretrained models, and frameworks such as Grounded SAM. Changes in library versions or the unavailability of model files may affect implementation and reproducibility.

Required versions and dependencies will be recorded. Important pretrained weights and configuration files should be maintained with the project where their licenses permit local storage.

4.10.7 Schedule Risk

Training, model tuning, and evaluation may require more time than initially expected. Failed experiments may further delay the comparison of the expert models and the complete framework.

Development will be performed incrementally. Initial work will focus on a small but complete pipeline before increasing dataset sizes or introducing additional experiments.

4.11 Conclusion

This chapter defined the requirements of the proposed research oriented image colorization prototype. The main functional requirements cover semantic region decomposition, contextual region extraction, expert routing, domain specific colorization, soft mask blending, evaluation, and expert extensibility.

The quality requirements focus on correctness, modularity, reproducibility, evaluability, maintainability, and resource awareness. The assumptions define the datasets, pretrained components, and scope required for the initial implementation. The hardware and software requirements describe the environment needed to conduct the experiments, while the interface and data organization sections explain how the prototype will be operated and how its experimental outputs will be maintained.

The identified risks demonstrate that the final performance of the system depends on several connected stages. For this reason, segmentation, routing, expert colorization, and blending will be evaluated separately as well as through the final generated image. These requirements provide the foundation for implementing and experimentally evaluating the proposed Mixture of Experts colorization framework.

Chapter 5 Proposed Approach and Methodology

This chapter presents the proposed approach as a framework for region-based Mixture-of-Experts (MoE) image colorization. Instead of predicting the colors of an entire photograph with a single network, the framework decomposes a grayscale image into semantic regions, assigns each region to a domain-specialized expert, and merges the expert outputs into one color image. The chapter first gives an overview of the complete pipeline (Section 5.1) and then describes each stage in turn: region decomposition (Section 5.2), expert routing (Section 5.3), context-aware region extraction (Section 5.4), expert colorization models (Section 5.5), blending (Section 5.6), and the procedure for adding new experts (Section 5.7). Section 5.8 closes the chapter with the evaluation protocol used to test each objective. The chapter covers the algorithmic and procedural details of the method; the system structure and component interfaces are covered in Chapter 6.

5.1 Overview of the Proposed Framework

The framework takes a grayscale photograph as input and produces a color photograph as output. It treats colorization as a divide-and-conquer problem. A single network must approximate the colors of portraits, landscapes, urban scenes, and animals at the same time, and these domains have different color distributions. The proposed framework therefore assigns each part of the image to a model trained for that kind of content.

5.1.1 Pipeline Stages

The pipeline consists of five stages, shown in Figure 5.1.

1. **Region decomposition.** A pretrained open-vocabulary segmentation model, Grounded-SAM [2], is prompted with a predefined set of text prompts and returns a mask for each detected region.
2. **Expert routing.** Each region is assigned to one of four domain experts (portraits and people, nature and landscapes, urban and man-made scenes, animals) according to the prompt that produced it.
3. **Context-aware region extraction.** The region is cropped together with an added boundary of approximately 20% (this value is variable, not fixed to allow for testing) around it, so the expert can see the surrounding context at the region edges.
4. **Expert colorization.** The assigned expert predicts the chrominance of the region from its luminance.

5. **Blending.** The expert outputs are merged into a single image using soft mask boundaries to avoid visible seams.

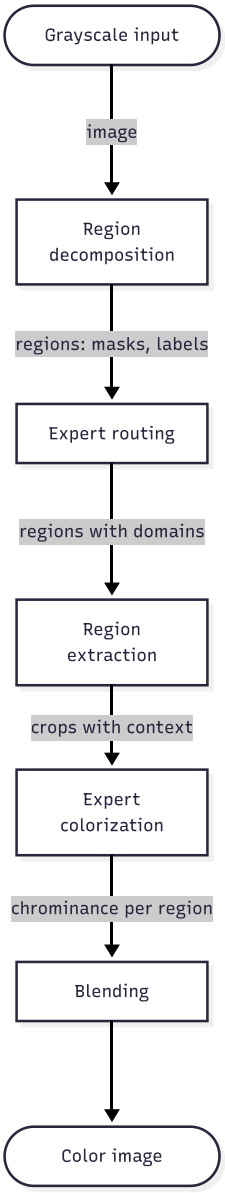


Figure 5.1: Overview of the proposed region-based Mixture-of-Experts colorization pipeline.

5.1.2 Color Space and Problem Formulation

All colorization is performed in the CIE Lab color space. Let $L \in \mathbb{R}^{H \times W}$ denote the luminance channel of the input image. The task is to predict the chrominance channels $ab \in \mathbb{R}^{H \times W \times 2}$. Because the grayscale input already contains the luminance, the framework predicts only ab and keeps the original L unchanged in the final image. This guarantees that the structure and brightness of the input photograph are preserved exactly, and that only color is added.

Let $\{M_k\}_{k=1}^{K}$ be the soft masks of the K regions found by the decomposition stage, where $M_k \in [0,1]^{H \times W}$

(Section 5.6). Each region is associated with an expert $E_{d(k)}$, where $d(k)$ is the domain assigned to region k by the routing stage. The pixels that belong to no region form an additional fallback region with index $k = 0$, whose mask is M_0 and which is colorized by the fallback expert E_0 (Section 5.3.3). The expert of region k predicts chrominance $\widehat{ab}_k$, and the final chrominance is the mask-weighted combination of the expert outputs over $k = 0, \ldots, K$ (described in Section 5.6):

$$\widehat{ab}(x) = \frac{\sum_{k=0}^{K} M_k(x)\, \widehat{ab}_k(x)}{\sum_{k=0}^{K} M_k(x)}, \tag{5.1}$$

where $k = 0$ is the fallback region (Section 5.3.3), and the output image is obtained by converting $(L, \widehat{ab})$ from Lab to RGB.

5.1.3 Relation to Existing MoE Approaches

Existing MoE work applies the idea at other levels. RS-MoEGAN [20] routes tokens inside a Vision Transformer, Gamma [19] uses shared and adaptive experts to handle dataset-level biases, and the product-key approach of He [18] scales to a very large number of small experts. In contrast, the proposed framework performs routing at the level of image regions, using a pretrained segmentation model as the router. As a result, no router needs to be trained, each expert only needs to learn a single domain, and a new expert can be added by defining a new prompt set (Section 5.7).

5.2 Region Decomposition

The first stage of the pipeline splits the grayscale input into semantically coherent regions. Region quality directly limits the final result, since a region that is missed, merged with a neighbor, or cut incorrectly is colorized by the wrong expert or with the wrong context. For this reason the decomposition stage is the component examined most closely in the evaluation (Section 5.8).

5.2.1 Segmentation Model

Region decomposition uses Grounded-SAM [2], which combines two pretrained models. An open-vocabulary detector (Grounding DINO [3]) takes an image and a text prompt and returns bounding boxes for the objects the prompt names. A promptable segmentation model (the Segment Anything Model, SAM [4]) then takes each box and returns a pixel-level mask. Both models are used as released, without fine-tuning. Because the detector accepts free-form text, the set of recognizable regions is defined by the prompt list and not by a fixed label set, which is what allows new experts to be added without changing the segmentation model (Section 5.7).

5.2.2 Domain Prompt Set

Each expert domain has a predefined list of text prompts built into the system, so no manual input is required from the user. Table 5.1 gives the initial prompt set. The set is a starting point and is refined during development by checking which prompts produce reliable masks on the evaluation data.

Table 5.1: Initial prompt set for each expert domain.

Domain	Prompts
Portraits and people	person, face
Nature and landscapes	sky, vegetation, tree
Urban and man-made scenes	buildings
Animals	animals

5.2.3 Procedure

For an input image the decomposition stage proceeds as follows.

1. **Load and resize.** The stored image is read as a single-channel grayscale image. If its longer side exceeds 1024 pixels, it is downscaled to that size with Lanczos resampling, keeping the aspect ratio. This limit keeps the memory use of the two models within the available GPU.

2. **Build the text prompt.** The class prompts of the domain prompt set are joined into a single lowercase string, with entries separated by a period and the string ending in a period. Grounding DINO receives this string once per image, so all classes are searched for in a single forward pass.

3. **Detect regions.** Grounding DINO returns a bounding box in pixel coordinates, a confidence score, and a text label for each detection. A detection is kept only if its box confidence exceeds the box threshold and its match to a word in the prompt exceeds the text threshold.

4. **Filter detections.** Detections whose score is below the minimum score are discarded. Duplicate boxes are then removed with per-class non-maximum suppression. Boxes are visited in order of decreasing score, and any remaining box with the same label whose intersection-over-union (IoU) with the visited box exceeds the suppression threshold is removed. Suppression is applied only within a class, so overlapping boxes with different labels are both retained.

5. **Segment each region.** The Segment Anything Model encodes the image once. Each retained box is then passed to it as a box prompt, and the model returns a single binary mask for that box. Because the box fixes where the object is, the segmentation model only refines the object boundary and does not search for objects itself.

6. **Merge fragments of the same object.** Masks are visited in order of decreasing area. For each

mask not yet merged, every other unmerged mask with the same label is absorbed if its containment exceeds the merge threshold, where containment is the area of the intersection divided by the area of the candidate mask. The absorbed mask is combined with the group by union, the group keeps the highest score among its members, and its bounding box is recomputed from the merged mask. The merge decision uses containment: the fraction of a smaller mask that lies inside a larger mask of the same label. A small fragment that lies almost entirely inside a larger mask has a containment close to one and is merged into it, even though it covers only a small part of the larger mask. Two separate instances of the same class, such as two people, overlap very little, so their containment is low and they remain separate regions.

7. **Output.** The stage returns a list of regions. Each region holds a binary mask, a label (the prompt that produced it), a confidence score, and a bounding box. The routing stage uses the label (Section 5.3) and the extraction stage uses the bounding box (Section 5.4).

The parameters of this stage are the maximum image side, the box threshold, the text threshold, the minimum detection score, the suppression threshold, and the merge threshold. Their values are selected on a tuning split of the segmentation dataset, a random subset of images drawn from its official training split and not used for any reported result. The resulting decomposition is compared against the ground-truth labels of this tuning split, and the parameters are then fixed before the segmentation test set (Section 5.8) is used.

5.2.4 Limitations of the Decomposition Stage

Three situations require handling after segmentation. Different prompts can produce overlapping masks for the same object (for example, *person* and *face*). Pixels covered by no mask need a defined fallback. A detection can also be wrong, either by naming the wrong object or by missing one. The rules for overlapping masks and uncovered pixels are part of the routing stage and are described in Section 5.3. The effect of detection errors on the final image is measured with the routing and end-to-end metrics in Section 5.8.

5.3 Expert Routing

After decomposition, each region must be assigned to the expert that will colorize it. Routing in the proposed framework is a fixed lookup from the region label to a domain. No router network is trained, so the assignment is deterministic and the only learned components are the experts themselves. A consequence is that routing quality depends on the detector and on the mapping together, and this combined quality is what the routing accuracy metric in Section 5.8 measures.

5.3.1 Label-to-Domain Mapping

Every prompt in the domain prompt set belongs to exactly one domain, so the label of a region (the prompt that produced it) determines its expert. Table 5.2 gives the mapping for the prompt set used in the prototype. Extending the framework to a new domain adds rows to this table (Section 5.7).

Table 5.2: Mapping from region labels to domain experts.

Label	Domain expert
person	Portraits and people
face	Portraits and people
sky	Nature and landscapes
vegetation	Nature and landscapes
tree	Nature and landscapes
building	Urban and man-made scenes
animal	Animals

A detection whose label does not match exactly one entry of the prompt set cannot be routed and is discarded. Its pixels are then treated as uncovered (Section 5.3.3).

5.3.2 Handling Overlapping Regions

Regions with different labels can overlap, for example a person standing in front of a building, or a tree against the sky. Regions with the same label that overlap are already merged during decomposition (Section 5.2). For the remaining overlaps, each contested pixel is assigned to a single region so that its color comes from one expert.

The rule is that a pixel belongs to the smallest region that contains it. Writing $|M_k|$ for the area of mask M_k, the owner of pixel x is

$$k^*(x) = \operatorname*{arg\,min}_{k:\,M_k(x)=1} |M_k|, \tag{5.2}$$

with ties broken by the higher detection score. Smaller regions are usually objects in front of a larger background, so this rule gives the foreground object priority over the surface behind it. The rule uses only mask areas and not the labels, so it applies unchanged when new experts are added. After ownership is assigned, each mask is reduced to the pixels it owns, which makes the masks disjoint. Soft boundaries are then created from these disjoint masks in the blending stage (Section 5.6).

5.3.3 Handling Uncovered Pixels

Some pixels belong to no region, either because no prompt matched them, because a detection was filtered out, or because a label could not be routed. To make sure every pixel receives a color, the framework includes a fallback expert, which is the pretrained colorization model used as released, without

domain fine-tuning. The fallback is treated as an additional region $k = 0$ whose mask contains exactly the uncovered pixels. It is run on the full image when at least one pixel is uncovered, and its output is used only inside its own mask. If every pixel is covered, the fallback is not run. The fallback region takes part in blending (Section 5.6) in the same way as the domain experts, so the combination in Section 5.1.2 sums over $k = 0, \ldots, K$.

5.4 Context-Aware Region Extraction

Each region produced by the decomposition stage and assigned an expert by the routing stage is next prepared as input for that expert. This stage cuts the region out of the image together with some of its surroundings, resizes the result to the input size of the expert network, and, after colorization, places the expert's output back at the correct position in the full image.

5.4.1 Cropping with Boundary Context

For a region with bounding box (x_0, y_0, x_1, y_1), width $w = x_1 - x_0$ and height $h = y_1 - y_0$, the crop is the rectangle

$$\big[x_0 - \rho w,\, x_1 + \rho w\big] \times \big[y_0 - \rho h,\, y_1 + \rho h\big], \tag{5.3}$$

clipped to the image boundaries, where $\rho \geq 0$ is the boundary fraction. The crop contains the region and a margin of ρ times the region's width and height on every side.

The boundary fraction is a configurable parameter of the pipeline and not a fixed constant. Its default value is $\rho = 0.2$, which corresponds to the margin of approximately 20% specified in the project proposal. Keeping it adjustable allows the effect of the margin on the final result to be tested (Section 5.8). A larger margin gives the expert more surrounding context, but it also makes the crop contain more pixels of neighboring regions, which increases the computation per region and reduces the resolution available to the region itself after the crop is resized to the expert's input size. A smaller margin has the opposite effects. The value is therefore chosen from experiments and not assumed.

The margin serves two purposes. It gives the expert the pixels around the region edge, so the predicted colors near the boundary are informed by what lies next to it. It also provides the extra pixels needed for the soft boundaries of the blending stage, because the soft mask extends slightly beyond the binary mask (Section 5.6). The margin must therefore remain wider than the smoothing width of the soft mask, and this sets a lower limit on ρ.

The bounding box is the one recorded by the decomposition stage, so a region that was merged from several fragments is cropped as a single unit. The fallback region (Section 5.3.3) is not cropped. The

fallback expert receives the whole image.

5.4.2 Input Resizing and Mask Handling

The expert networks expect a fixed square input, whereas crops have arbitrary size and aspect ratio. The luminance of each crop is therefore resized to a square of side S, where S is the input size of the expert network, using bicubic interpolation. The expert predicts the chrominance at this size.

The region mask is not given to the expert as an input. The expert receives the full crop, including the margin and any pixels of neighboring regions inside the rectangle, and colorizes all of it. The mask is applied afterwards, when the output is placed back into the image (Section 5.4.3), so that only the pixels inside the region are taken from this expert. Two reasons support this choice. First, the pretrained network and its training code are used without architectural changes, which keeps the experts compatible with the released weights. Second, the expert sees natural image content throughout the crop and not an image with masked-out areas, which is closer to the data it was trained on.

5.4.3 Mapping Expert Outputs Back to the Image

The chrominance predicted for a crop is resized from $S \times S$ back to the size of the crop, again with bicubic interpolation, and written into a canvas of the full image size at the position of the crop. Pixels of the canvas outside the crop have no prediction from this expert. All coordinates in this stage refer to the image after the size limit applied in decomposition (Section 5.2). The resulting per-region chrominance maps, together with the region masks, are the inputs of the blending stage.

5.5 Expert Colorization Models

Each of the four domains has its own colorization expert. The experts share one architecture and differ in their weights, which are trained separately on the data of one domain each. This section describes the architecture, the training data, the training procedure, and the way an expert is applied at inference time.

5.5.1 Base Architecture

All experts are instances of DDColor [14], a colorization network that predicts the chrominance of an image from its luminance. DDColor consists of a backbone encoder that extracts multi-scale features from the grayscale input, a pixel decoder that restores spatial resolution, and a color decoder, a transformer that uses a set of learnable color queries to attend to the image features and learn semantic color representations. The outputs of the two decoders are fused to give the *ab* channels of the Lab color

space. This design matches the framework's formulation in Section 5.1.2, since the network takes L and predicts ab.

The tiny variant of DDColor (ConvNeXt-T backbone, 55.0 million parameters) is used. Four copies of this variant total approximately 220 million parameters, close to the size of the largest published variant (227.9 million), which allows a comparison against a single large generalist with a similar parameter budget (Section 5.8). Each expert is initialized from the same pretrained checkpoint and then trained independently, so no weights are shared between experts. This independence is what allows an expert to be added or replaced without affecting the others (Section 5.7). An alternative design, a frozen shared backbone with small per-domain adapters, was considered as it would reduce storage, however it was ultimately rejected as it would couple the experts through the shared weights.

The experts operate at a fixed square input size S, taken from the configuration of the pretrained network ($S = 256$ by default). The resizing of region crops to this size is described in Section 5.4.

5.5.2 Training Data

Each expert is trained on whole color images from its own domain. Grayscale inputs are derived from these images by converting them to the Lab color space and taking the L channel as the input and the ab channels as the target, so no manual annotation is needed. Table 5.3 lists the public datasets identified for each domain.

Table 5.3: Training datasets for each expert domain.

Domain	Datasets
Portraits and people	CelebA, CelebA-HQ, FFHQ (faces); DeepFashion-MultiModal, COCO person images (full body)
Nature and landscapes	Places365, SUN397
Urban and man-made scenes	Cityscapes, ADE20K, urban subsets of Places365
Animals	AFHQ, Oxford-IIIT Pets, animal subsets of COCO

For datasets that cover many scene types (Places365, SUN397, ADE20K, COCO), only the scene categories or images that belong to the expert's domain are used. Each domain is split into training, validation, and test images, and the test images are never used for training or for selecting hyperparameters (Section 5.8). Face datasets are included for the portrait expert because they provide many images with a consistent subject, and full-body datasets are included so that the expert also sees the larger and more varied people regions that the decomposition stage produces.

5.5.3 Training Procedure

Each expert is trained by fine-tuning the pretrained DDColor-T checkpoint on the training images of its domain (Table 5.3), using the training code released with DDColor. Every training image is converted to the Lab color space and resized to the network input size $S \times S$. The luminance channel L is the input of the network and the chrominance channels ab are the target, so no annotation is needed.

The training objective is the one used by the original network, a combination of a pixel-level loss, a perceptual loss, an adversarial loss, and a colorfulness loss, and it is not modified. The experts are trained one at a time with identical settings, so that differences between the experts come from the training data and not from the training configuration. Each expert is initialized from the same pretrained checkpoint, and no weights are shared or exchanged between experts during training.

The validation images of each domain are used to monitor training and to select the final checkpoint. The test images are never used for training or for any selection (Section 5.8). The learning rate, batch size, number of iterations, and loss weights are given with the experimental results. The fallback expert (Section 5.3.3) is not trained. It uses the pretrained checkpoint as released.

5.5.4 Inference Procedure

At inference time, the experts are used only for prediction. For each region, the pipeline loads the expert assigned by the routing stage, passes it the luminance of the resized crop, and receives the predicted chrominance at size $S \times S$. The result is resized back to the crop size and placed into the image (Section 5.4.3). Only the experts needed for the regions found in an image have to be in memory, so experts can be loaded when first needed and released afterwards. The fallback expert (Section 5.3.3) uses the pretrained checkpoint without fine-tuning and processes the whole image in a single pass.

5.6 Blending

The extraction stage produces one chrominance map per region, each covering only its own crop (Section 5.4.3). The blending stage combines these maps into a single chrominance map for the whole image, and then recombines it with the original luminance to give the final color image. The stage has two aims: neighboring regions colorized by different experts should join without a visible seam, and the output should keep the full detail of the input photograph.

5.6.1 Soft Mask Construction

After overlap resolution (Section 5.3.2) the regions are disjoint, and together with the fallback region (Section 5.3.3) they cover every pixel of the image exactly once. Let $B_k \in \{0,1\}^{H\times W}$ be the binary mask of region k, for $k = 0,\ldots,K$, where $k = 0$ is the fallback region. A binary mask would give a hard switch from one expert to the next at the region boundary, so each mask is softened with a Gaussian filter:

$$M_k = \left(G_{\sigma_k} * B_k\right) \cdot \mathbf{1}_{C_k}, \tag{5.4}$$

where G_{σ_k} is a Gaussian kernel with standard deviation σ_k and $\mathbf{1}_{C_k}$ restricts the result to the crop rectangle C_k of the region (for the fallback region, C_0 is the whole image). The soft mask M_k takes values in $[0,1]$. It is close to one well inside the region and falls smoothly to zero across the boundary.

An expert has predictions only inside its crop, so the soft mask must not extend beyond it. A Gaussian filter has most of its support within about $3\sigma_k$ of the boundary, and the crop margin is ρw_k and ρh_k on the sides (Section 5.4). The smoothing width of each region is therefore limited by the size of its margin:

$$\sigma_k = \min\left(\sigma_0,\ \frac{\rho \min(w_k, h_k)}{3}\right), \tag{5.5}$$

where σ_0 is the base smoothing width, a configurable parameter of the pipeline. Small regions, which have narrow margins, are smoothed less than large regions.

5.6.2 Weighted Chrominance Combination

Let $\widehat{ab}_k$ denote the chrominance predicted by the expert of region k, placed on a canvas of the full image size and set to zero outside the crop. The combined chrominance is the normalized weighted average

$$\widehat{ab}(x) = \frac{\sum_{k=0}^{K} M_k(x)\,\widehat{ab}_k(x)}{\sum_{k=0}^{K} M_k(x)}. \tag{5.6}$$

Inside a region, far from its boundary, only that region's mask is nonzero, so the output equals the prediction of its expert. Near a boundary the masks of both neighboring regions are nonzero, and the output moves gradually from one expert's prediction to the other's. The denominator is positive at every pixel, because each pixel belongs to exactly one region, lies inside that region's crop, and receives a positive weight from its own region's soft mask. No division by zero occurs, and the normalization compensates for the masks not summing exactly to one after smoothing and clipping.

5.6.3 Recombination with the Original Luminance

The combined chrominance $\widehat{ab}$ has the size of the image processed by the earlier stages, which may be smaller than the stored input if the size limit of the decomposition stage was applied (Section 5.2). To restore the original resolution, the final image is formed in three steps.

1. If the image was downscaled, $\widehat{ab}$ is resized to the original width and height with bilinear interpolation. Chrominance varies slowly compared with luminance, so little detail is lost by interpolating it. If the image was not downscaled, this step is skipped.
2. The luminance channel L_0 is computed from the original full-resolution grayscale image. It is used unchanged, so the structure, texture, and brightness of the input are preserved exactly.
3. The channels $(L_0, \widehat{ab})$ are converted from Lab to RGB. Colors outside the RGB gamut are clipped to the valid range.

None of the networks processes the full-resolution image. They work at the reduced size, and only the low-frequency chrominance information is carried to the original resolution. The effect of the blending stage on the final image is measured by the color difference across region boundaries and by the quality metrics of the final image with and without blending (Section 5.8).

5.7 Extensibility: Adding a New Expert

The framework is designed so that a new domain can be added without redesigning the system or retraining the existing experts. Two properties of the design make this possible. Routing is a lookup from region label to domain (Section 5.3), so there is no router to retrain. Blending uses fixed filters with no learned parameters (Section 5.6), so there is nothing to retrain there either. The only learned component that a new domain requires is the new expert itself.

5.7.1 Procedure for Adding an Expert

A new expert is added in four steps.

1. **Define the domain and its prompts.** A set of text prompts is chosen for the objects and surfaces that the new expert should handle. Each prompt must belong to exactly one domain, so a new prompt must not duplicate or closely overlap an existing one (Section 5.3).
2. **Collect training data.** Whole color images of the new domain are gathered from public datasets and split into training, validation, and test images, in the same way as for the existing domains (Section 5.5).

3. **Train the expert.** A new copy of the pretrained DDColor-T checkpoint is fine-tuned on the data of the new domain with the training procedure of Section 5.5.3. No existing expert takes part in this training.

4. **Register the expert.** The new prompts are added to the prompt set, the corresponding rows are added to the label-to-domain mapping (Table 5.2), and the path of the new checkpoint is added to the list of experts.

The first and last steps are changes to configuration and do not alter the decomposition models, the overlap rule, the fallback, or the blending. The segmentation models are used as released, so the new prompts need no additional segmentation training.

5.7.2 Independence of Existing Experts

The weights of each expert are stored in a separate checkpoint, and no parameters are shared between experts (Section 5.5). Adding an expert therefore leaves the checkpoints of the existing experts untouched, and this can be verified directly by comparing checksums of the checkpoint files before and after the addition.

Two indirect effects on the existing domains remain possible, and neither involves a change to an expert. First, all prompts are given to the detector as one text string (Section 5.2), so adding prompts changes that string, which may change the detections obtained for the existing prompts. Second, regions of the new domain can overlap regions of existing domains, and the overlap rule (Section 5.3.2) can then assign some pixels to a different owner than before. Both effects are measured in the extensibility test (Section 5.8), by comparing the regions and the final colorization of images from the existing domains before and after the new expert is added.

5.8 Evaluation Protocol

This section describes how each objective of the project is tested. It defines the datasets and splits, the metrics, the baselines that the framework is compared with, the ablations that isolate the contribution of its components, and the test of extensibility.

5.8.1 Datasets and Splits

Four kinds of test data are used.

- **Domain test sets.** For each expert domain, the held-out test images of the domain datasets (Table 5.3). They are used to evaluate each expert on its own domain.

- **Region-crop test sets.** The region crops that the pipeline produces from the domain test images, using the decomposition and extraction stages. They are used to measure how the experts perform on the inputs they receive at inference, as opposed to whole images (Section 5.5.3).
- **Segmentation tuning and test sets.** The segmentation dataset (ADE20K or COCO) is used in two disjoint parts. The tuning split is a random subset of its official training split and is used to select the decomposition parameters (Section 5.2). The test set is its official validation split, which is not used for any selection and is evaluated only after all parameters are fixed. The test set is used to evaluate region decomposition and routing.
- **Mixed-scene test set.** Images that contain regions from at least two domains (for example a person in an urban scene), selected from the segmentation test set using its annotations.

The ground truth for colorization is the original color image, and the grayscale input is its L channel. Test images are never used for training or for choosing parameters. The detection and merge thresholds are chosen on the segmentation tuning split. The smoothing width σ_0 and the boundary fraction ρ are chosen on the validation images of the domain datasets. Where two datasets contain the same image, the duplicate is removed from the training data before the test images are fixed.

5.8.2 Region Decomposition Metrics

The quality of decomposition is measured with the mean intersection-over-union (mIoU) on COCO-Stuff [5], which provides both object (thing) and background (stuff) classes, such as sky, grass, tree, water, and road, for the same images. Plain COCO instance annotations contain only object classes and cannot evaluate the nature and urban prompts. Each ground-truth class is mapped to the prompt label that covers it. The *animal* prompt covers all of the animal classes of the dataset (for example dog, cat, bird, horse, and cow), and prompts that name closely related categories, such as *vegetation* and *tree*, are evaluated together as one group. The prompt *face* is evaluated as part of the person group, since COCO-Stuff has no face class. Classes without a matching prompt are ignored. Every pixel of a test image is given a predicted group, which is the group of the label of the region that owns it after overlap resolution (Section 5.3.2), or an *uncovered* label if no region owns it. The IoU of each group is computed between the predicted and ground-truth label maps, and mIoU is the mean over the groups. It is reported for each group, for each domain (by merging the groups of a domain), and overall. The fraction of pixels owned by some region is reported with it, because this fraction determines how much of an image is colorized by the fallback expert rather than a domain expert.

5.8.3 Expert Colorization Metrics

Each expert is evaluated on the test set of its own domain, with the following metrics.

- **FID** [28]: the distance between the feature distributions of the colorized images and the original color images. Lower is better. FID is unreliable on small sets, so the test sets must contain enough images for a stable estimate, and the number of images is reported with each value.
- **LPIPS** [29]: a learned perceptual distance between each colorized image and its ground truth. Lower is better.
- **Colorfulness** [30]: a measure of how saturated and varied the colors of an image are, computed without a reference. It is reported next to the other metrics to detect outputs that are desaturated, which is a typical failure of colorization models.
- **PSNR and SSIM**: pixel-level and structural similarity to the ground truth. Because the luminance of the output equals that of the input, differences come from color only.
- ΔE: the mean Euclidean distance in the Lab color space between the output and the ground truth. As the luminance is identical, this is the error in the chrominance channels.

A grayscale photograph has many plausible colorings, so similarity to the one original coloring (PSNR, SSIM, ΔE) penalizes plausible alternatives. These metrics are therefore read together with FID, LPIPS, and colorfulness and not alone. Each expert is evaluated on the whole-image test set and on the region-crop test set, and the difference between the two shows how much the use of whole images in training affects the results on crops.

5.8.4 Routing Accuracy

Routing accuracy is the fraction of regions that are assigned to the correct expert. For each region produced on the segmentation test set, the correct expert is determined from the ground truth. The ground-truth class that covers the largest share of the region's pixels is found, and the domain of that class is taken from the class-to-domain mapping. A region is routed correctly if its assigned domain equals this domain. Regions whose dominant class belongs to no domain are excluded from the accuracy and counted separately. The accuracy is reported overall and for each domain, with a confusion matrix between domains. Since routing is a lookup from the label (Section 5.3), this metric measures the combined correctness of the detector's labels and of the mapping.

5.8.5 Blending Metrics

Seams are measured by the color discontinuity across the boundary between two regions that were colorized by different experts. For each such boundary, the mean ΔE is computed between pixels on opposite sides of the boundary, at a fixed small distance from it. The same quantity is computed on the ground-truth image, because real images also change color at object boundaries, and the difference between the two values is reported as the excess discontinuity introduced by the pipeline. In addition, the FID and LPIPS of the final images are computed with and without the blending module, where the variant without blending uses the binary masks to paste the expert outputs side by side.

5.8.6 Baselines and Ablations

The framework is compared with three baselines, all evaluated on the single-domain test sets and on the mixed-scene test set.

1. **Pretrained DDColor-T**, applied to the whole image as released.
2. **DDColor-T fine-tuned on pooled data**: one network fine-tuned on the training images of all four domains together, with the same training settings. This separates the effect of specialization from the effect of fine-tuning on the domain data.
3. **Pretrained DDColor-L**, the largest released variant, applied to the whole image. Its size (227.9 million parameters) is close to that of the four experts together (about 220 million), so it provides a comparison at a similar parameter count.

Table 5.4 lists the ablations, each of which changes one component of the pipeline and keeps the rest unchanged.

Table 5.4: Ablations of the proposed framework.

Component	Variants	Measured by
Blending	on (soft masks); off (binary masks pasted)	boundary ΔE, FID, LPIPS
Smoothing width σ_0	several values	boundary ΔE, FID, LPIPS
Boundary fraction ρ	0, 0.1, 0.2, 0.3	boundary ΔE, FID, LPIPS
Routing	correct routing; deliberately shuffled routing	FID, LPIPS, ΔE
Region shape	segmentation masks; bounding boxes only	boundary ΔE, FID, LPIPS

The shuffled-routing variant assigns each region to a randomly chosen wrong expert. It shows how much of the result depends on routing correctly, and therefore how large the loss from routing errors can be.

5.8.7 Extensibility Test

The extensibility claim is tested by adding a fifth expert for the domain Vehicles, following the procedure of Section 5.7. The test checks three things.

1. **Existing experts are unchanged.** The checksums of the four existing checkpoints are compared before and after the addition.
2. **The new expert works.** It is evaluated on the test set of its domain with the metrics of Section 5.5, against the baselines above.
3. **Existing domains are not degraded.** For images from the four existing domains, the regions found by the decomposition stage and the final results are compared before and after the addition. The change in the regions is measured by the IoU between the masks before and after, and the change in the results by the difference in FID and LPIPS. Any difference can arise only from the changed prompt string and from the overlap rule (Section 5.7), and the test reports its size.

The files and configuration entries that had to be changed to add the expert are also recorded, to document how much of the system was left untouched.

Conclusion

This chapter described a region-based Mixture-of-Experts framework for image colorization. A pre-trained open-vocabulary segmentation model decomposes the grayscale input into regions, each region is routed to an expert by a lookup from its label, and the experts, which are fine-tuned copies of DDColor-T with independent weights, predict the chrominance of the crops they receive. The outputs are merged with soft masks and combined with the original luminance, so that the detail of the input is preserved. A new expert is added by defining new prompts and training one new network, without modifying the existing ones. The evaluation protocol tests each of these claims against baselines and ablations. The next chapter presents the architecture of the system and the design of its components.

Chapter 6 High-Level and Low-Level Design

This chapter presents the design of the system that realizes the method described in Chapter 5. It begins with an overview of the system and its basic design approach (Section 6.1). It then states the assumptions, constraints, and goals that shaped the design (Section 6.2). The architecture is described next, from the top-level components down to the subsystems and their interactions (Section 6.3), followed by the strategies behind the main design decisions (Section 6.4). The low-level design is given as a class diagram with descriptions of its classes (Section 6.5), and the chapter closes with the policies and tactics that govern implementation details such as configuration, testing, and source organization (Section 6.6).

6.1 System Overview

6.1.1 Purpose and Functionality

The system converts a grayscale photograph into a color image. It is the working prototype of the region-based Mixture-of-Experts colorization framework proposed in this project. Instead of colorizing the whole image with one network, the system divides the image into semantic regions, colorizes each region with an expert trained for its domain, and merges the results into one image.

At run time, the system performs the following functions:

- **Region decomposition:** detects and segments the regions of the image that belong to the supported domains.
- **Expert routing:** assigns each region to a domain expert, resolves overlaps between regions, and provides a fallback for pixels that belong to no region.
- **Region extraction:** cuts each region out together with its surrounding context and prepares it as input for its expert.
- **Expert colorization:** predicts the chrominance of each region with the assigned expert.
- **Blending and reconstruction:** merges the per-region predictions with soft masks and combines the result with the original luminance to form the output image.
- **User interaction:** lets the user upload a grayscale photograph through a basic web interface and view the colorized result.

Two further functions take place outside this path. The experts are trained offline on domain datasets, and the system as a whole is evaluated against baselines with the metrics of Section 5.8. The system is also designed so that a new expert can be added as an extension (Section 5.7).

The scope of the system is photographic colorization of still images. It does not provide manual control over the colors of individual objects, and extension to video and medical imagery is outside the present work.

6.1.2 Basic Design Approach

The system is organized as a pipeline of stages, and each stage is implemented as a subsystem with a defined input and output. A stage receives the output of the previous stage and does not depend on how that output was produced. This keeps the stages separable, so that each one can be tested on its own and replaced without changes to the others. Table 6.1 relates the stages of the method in Chapter 5 to the subsystems of the design.

Table 6.1: Relation between the stages of the method (Chapter 5) and the subsystems of the design.

Stage	Chapter 5	Subsystem
Region decomposition	Section 5.2	Region Decomposition
Expert routing	Section 5.3	Routing
Region extraction	Section 5.4	Region Extraction and Expert Colorization
Expert colorization	Section 5.5	Region Extraction and Expert Colorization
Blending and reconstruction	Section 5.6	Blending and Reconstruction
Image upload and result display	—	User Interface
Training and evaluation	Sections 5.5.3, 5.8	Training and Evaluation (offline)

Three decisions define the design. First, only the experts are learned. The segmentation models are used as released, routing is a lookup, and blending uses fixed filters, so training is limited to the four expert networks. Second, the experts share one architecture and have independent weights, which allows an expert to be added or replaced without affecting the others. Third, the user interface is kept separate from the pipeline. The interface passes an image to the pipeline and displays the result, and it contains no colorization logic. The reasons for these decisions are given in Section 6.4.

6.1.3 Inputs, Outputs, and Users

The system is used through a basic web interface. The user uploads a grayscale photograph, which is a stored image file in a common format such as PNG, and the system returns the colorized image, which is displayed in the interface and can be saved. The user supplies no text prompts, because the prompts for each domain are built into the system. For evaluation, the grayscale input is obtained from the luminance channel of a color photograph, so that the original serves as the ground truth.

The output is a color image of the same size as the input.

The interface is deliberately minimal. It provides image upload, a way to start the colorization, and display of the result. It offers no manual control over the colors or regions of the image. The intended users are people who want to colorize a photograph, and they need no knowledge of the underlying models.

6.2 Design Considerations

This section describes the assumptions, constraints, goals, and development approach that were settled before the design was fixed.

6.2.1 Assumptions and Dependencies

- **Pretrained models.** The system depends on three pretrained models that are used as released: Grounding DINO and the Segment Anything Model for region decomposition, and the DDColor-T checkpoint as the starting point of every expert and as the fallback expert. It is assumed that these models remain available from their public sources and that their licenses permit their use in this project. Their versions are fixed so that results can be reproduced.
- **Software.** The system is written in Python and uses PyTorch and the libraries on which the pretrained models depend. Library versions are pinned, because the released training and inference code of the pretrained models was written for specific versions.
- **Execution environment.** The system is developed, trained, and run initially in free cloud notebook environments, each of which provides a single GPU of moderate memory for a limited time. These resources may prove insufficient for training four experts and running the complete pipeline, in which case a dedicated GPU system will be required. While the free environments are used, the web interface can be reached only while a session is running.
- **Datasets.** The public datasets listed in Table 5.3 remain downloadable, and their licenses permit research use.
- **Input images.** Inputs are grayscale photographs of the scene types covered by the four domains. Images of other kinds, such as drawings or documents, are not a target of the system, and any of their pixels that match no prompt are handled by the fallback expert.
- **Prompt language.** The text prompts are in English, since the detector is prompted with English text.
- **Expected changes.** New experts and prompts are expected to be added over time (Section 5.7), and the prompt set is refined during development (Section 5.2). The design therefore treats the

prompt set, the label mapping, and the list of experts as data and not as fixed code.

6.2.2 General Constraints

- **GPU memory.** A single GPU with limited memory runs all models. This is the reason for the size limit applied to images in the decomposition stage (Section 5.2) and for loading experts only when a region is routed to them (Section 5.5).
- **Session length and compute quota.** Sessions end after a limited time and the weekly GPU time is limited. Training an expert can therefore be interrupted, and the training of four experts has to fit within the available quota. The impact on the design is that training saves checkpoints regularly and can be resumed from them, that the smallest DDColor variant is used, and that the amount of training data and the number of training iterations are chosen to fit the budget.
- **Storage.** The disk space of the notebook environment is limited and, for part of the platforms, not kept between sessions. Large datasets are therefore used as subsets, and checkpoints and results are saved to persistent storage outside the session.
- **No permanent server.** While the system runs inside notebook sessions, there is no always-available server for the web interface, and the interface is a means of demonstrating the prototype and not a deployed service.
- **Dependence on segmentation quality.** The pipeline cannot colorize a region with a domain expert if the segmentation models do not find it. Their detection errors are treated as a limit of the system and are measured and not corrected (Section 5.8).
- **Dataset licenses.** The licenses of the datasets restrict their use to research purposes, so the trained experts are research artifacts and not products.
- **Performance requirements.** No real-time requirement is placed on the prototype. An image passes through several models, and the processing time is accepted as long as it allows the evaluation to be run.
- **Schedule.** The project has a fixed duration, which limits the number of domains, the dataset sizes, and the number of experiments.
- **Verification.** The system is verified by the evaluation of Section 5.8 and by the tests described in Section 6.6.

6.2.3 Goals and Guidelines

The following goals and guidelines shaped the design. When two of them conflict, they are applied in the order in which they are listed in the last paragraph of this subsection.

- **Preservation of the input.** The luminance of the input is kept unchanged in the output, and only chrominance is predicted. The reason is that the structure and brightness of the photograph are already known, and predicting them would add error without adding information. It also makes the comparison of results with the ground truth depend on color alone.
- **Extensibility.** A new expert can be added without retraining or modifying the existing ones (Section 5.7). This is Objective 5 of the project, and it is the reason for the independent weights of the experts and for treating the label mapping and the list of experts as data.
- **Modularity.** Each stage has a defined input and output and does not depend on the internals of the others. The reason is that the stages can then be tested separately and replaced, for example a different segmentation model or a different expert architecture, without changing the rest.
- **Reuse of pretrained components.** Existing pretrained models are used instead of training equivalents from scratch. The reason is the compute budget, which does not allow the training of a detector, a segmentation model, and four colorization networks from scratch.
- **Low memory use over speed.** Where memory and processing time conflict, memory is given priority, because the available GPU memory is the tightest resource. An example is loading experts when they are needed, which costs time on first use but avoids holding all experts in memory.
- **Reproducibility.** Library versions, model checkpoints, and random seeds are recorded, because notebook sessions are temporary and an experiment must be repeatable in a new session.
- **Simplicity.** Where a simple rule meets the need, it is preferred to a more complex one, for example a lookup for routing and a fixed rule for overlaps. The reason is that simple rules are easier to explain, test, and extend within the available time.

In order of priority, the design favors the quality of the colorization, then extensibility, then memory use, and then processing time.

6.2.4 Development Methods

The system is developed with an iterative model. The system is built in a sequence of iterations, and each iteration adds or refines one stage of the pipeline and ends with a test on a small set of images. The results of an iteration decide what the next one changes. The first iteration produced a standalone

prototype of region decomposition with Grounding DINO and the Segment Anything Model, whose behavior on test images determined the procedure of Section 5.2. The following iterations are planned in this order: routing and region extraction with the pretrained DDColor model as the only expert, fine-tuning of the four experts, blending and reconstruction, evaluation against the baselines, and finally the web interface and the extension with a fifth expert.

Two other approaches were considered. A waterfall model, in which the complete design is fixed before implementation, was rejected because the performance of the experts and the behavior of the segmentation stage cannot be known before they are tried, and the design has to respond to the results. A formal agile framework such as Scrum was not adopted, since its roles and ceremonies add process that a project of this size does not need. The iterative model takes from it the short cycles with a working result at the end of each one.

6.3 System Architecture

This section describes how the functionality of the system is divided among components, how the components cooperate, and how the system relates to the models and data outside it. The components follow the stages of the method in Chapter 5.

6.3.1 Architecture Overview

Figure 6.1 shows the top-level architecture. The system has a run-time part, which colorizes an uploaded image, and an offline part, which produces the trained experts and evaluates the system. The two parts are connected only through the expert checkpoints.

The run-time part consists of the following components.

- **User Interface:** receives the uploaded image and displays the result. It contains no colorization logic.
- **Pipeline Controller:** the single entry point of the pipeline. It keeps the original image, calls the subsystems in order, passes the output of each to the next, and returns the final image to the interface. The subsystems do not call each other.
- **Region Decomposition:** finds the regions of the image that belong to the supported domains.
- **Routing:** assigns each region to a domain, makes the regions disjoint, and creates the fallback region for uncovered pixels.
- **Region Extraction and Expert Colorization:** prepares each region as expert input, runs the

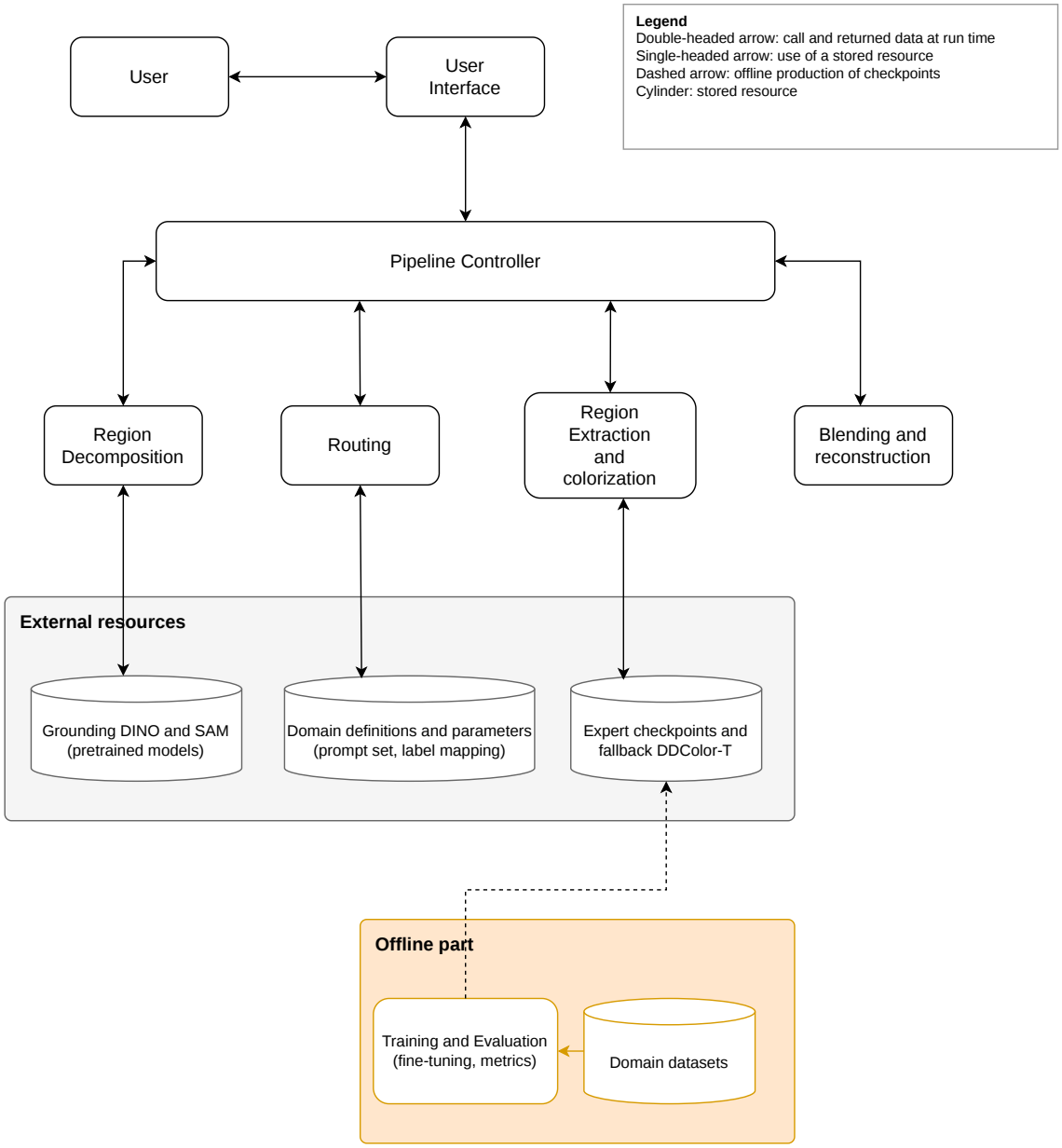


Figure 6.1: Top-level architecture of the system.

assigned expert, and places the prediction back into the image.

- **Blending and Reconstruction:** merges the predictions of all regions and combines them with the original luminance.

The offline part consists of the **Training and Evaluation** subsystem, which fine-tunes the experts and measures the results of the system.

External architecture. The system depends on three pretrained models that it does not modify: Grounding DINO and the Segment Anything Model, used by Region Decomposition, and the DDColor-T checkpoint, which is the starting point of the experts and the fallback expert. The domain datasets are used only by the offline part. Besides these, the subsystems read the domain definitions (the prompt set and the label-to-domain mapping) and the numerical parameters of the method. The subsystems are connected through the data they exchange, which is summarized in Table 6.2.

Table 6.2: Inputs and outputs of the run-time subsystems.

Subsystem	Input	Output
Region Decomposition	Grayscale image	Regions (mask, label, score, bounding box)
Routing	Regions	Disjoint regions with assigned domains, and the fallback region
Region Extraction and Expert Colorization	Routed regions, luminance	Chrominance map for each region
Blending and Reconstruction	Chrominance maps, masks, original luminance	Color image

6.3.2 Subsystem Architecture

6.3.2.1 Region Decomposition

The subsystem turns a grayscale image into a list of regions. Figure 6.2 shows its components.

- **Detector:** runs Grounding DINO on the image with the prompt set and returns boxes, scores, and labels.
- **Detection Filter:** removes detections below the minimum score and applies per-class non-maximum suppression.
- **Segmenter:** runs the Segment Anything Model with each retained box as a prompt and returns one binary mask per box.
- **Region Merger:** merges the masks of the same label that fragment one object, using the containment rule.

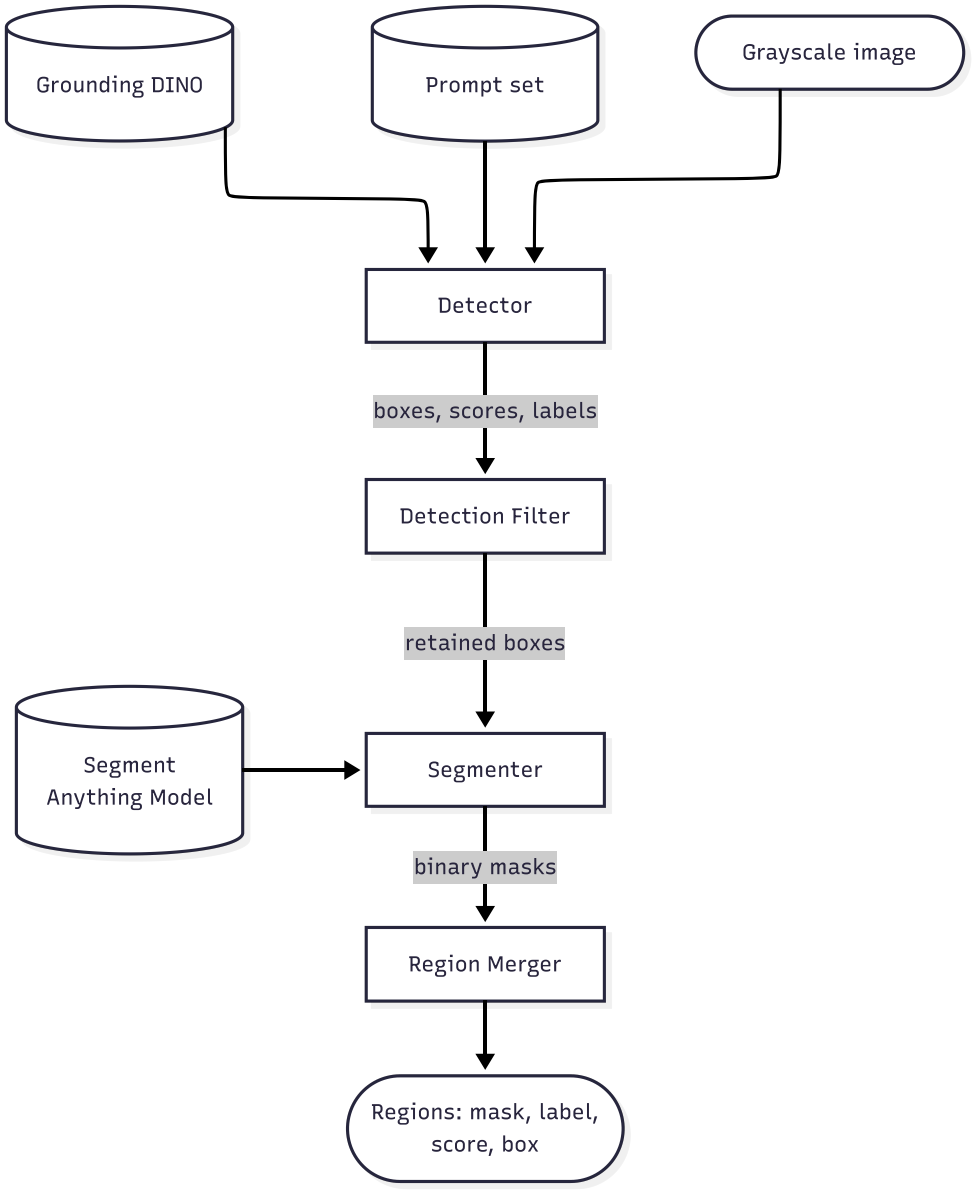


Figure 6.2: Components of the Region Decomposition subsystem.

The procedure that these components carry out is described in Section 5.2. The subsystem also limits the image size before detection and returns the scale factor, so that the later stages can restore the original resolution.

6.3.2.2 Routing

The subsystem prepares the regions for the experts. Its components are the **Label Mapper**, which assigns each region the domain of its label and discards regions with an unknown label, the **Overlap Resolver**, which gives each contested pixel to the smallest region that contains it, and the **Fallback Handler**, which builds the fallback region from the pixels that no region owns. The rules are given in Section 5.3.

6.3.2.3 Region Extraction and Expert Colorization

This subsystem produces the chrominance of each region. Figure 6.3 shows its components.

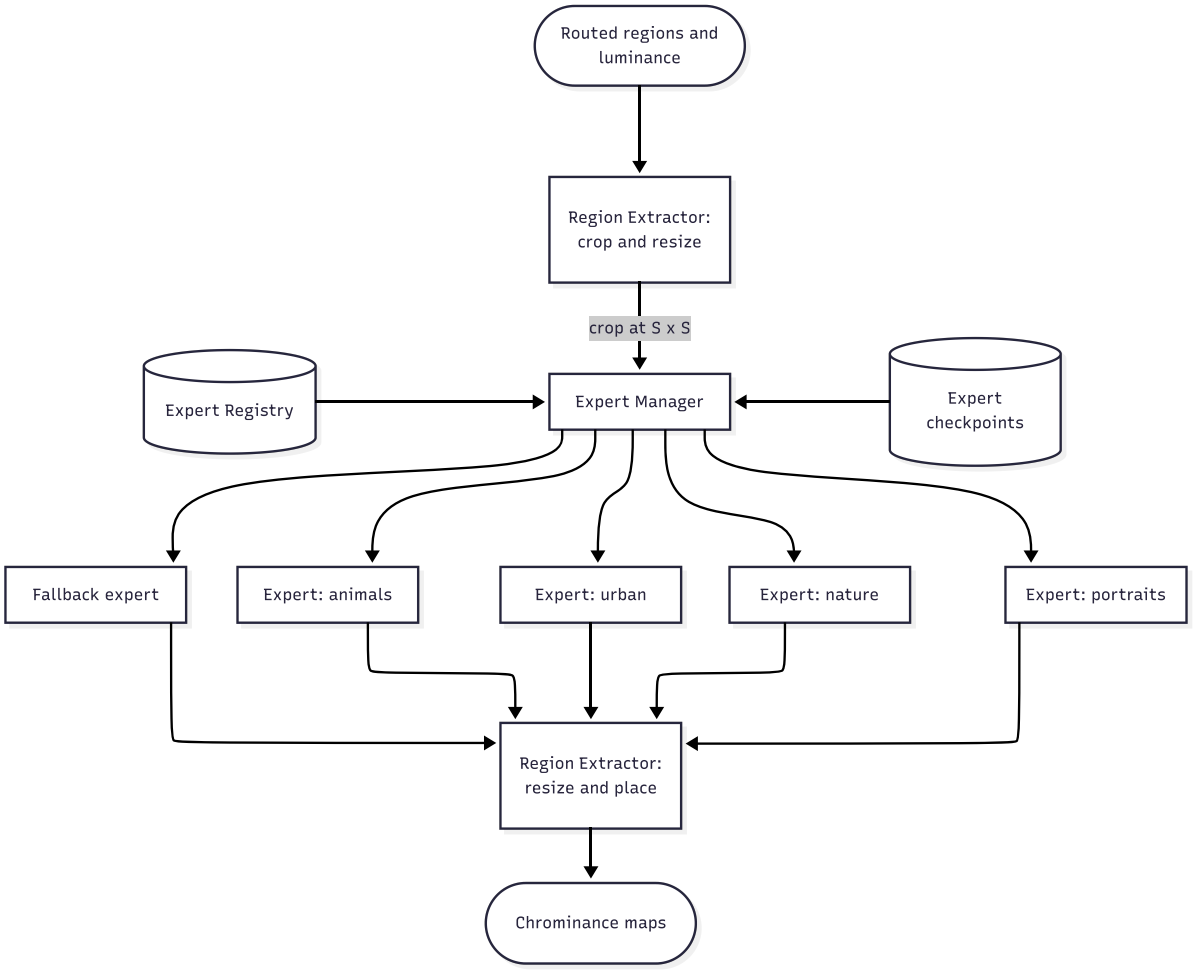


Figure 6.3: Components of the Region Extraction and Expert Colorization subsystem.

- **Region Extractor:** crops each region with its boundary context, resizes the crop to the expert

input size, and places the expert output back at the crop position (Section 5.4).

- **Expert Registry:** records which expert belongs to which domain and where its checkpoint is stored.
- **Expert Manager:** loads the checkpoint of an expert when a region is routed to it for the first time, and releases experts that are no longer needed.
- **Experts:** one DDColor-T instance for each domain, with independent weights.
- **Fallback Expert:** the pretrained DDColor-T checkpoint as released, which colorizes the fallback region from the whole image.

6.3.2.4 Blending and Reconstruction

The subsystem forms the final image from the per-region predictions (Figure 6.4). The **Soft Mask Builder** smooths the region masks, the **Chrominance Combiner** computes the normalized weighted average of the predictions, and the **Luminance Recombiner** resizes the combined chrominance to the original size, joins it with the original luminance, and converts the result to RGB (Section 5.6).

6.3.2.5 User Interface

The interface is a basic web page with an upload control, a control that starts the colorization, and a display of the result. It sends the image to the Pipeline Controller and shows the returned image. Because it only exchanges images with the controller, the pipeline can be run without it, which the evaluation scripts do.

6.3.2.6 Training and Evaluation

This offline subsystem is carried out by the developers and takes no part in the processing of an uploaded image. The developers build the training, validation, and test splits of each domain and convert the images to the Lab color space. They then fine-tune a copy of the pretrained DDColor-T checkpoint for each domain (Section 5.5.3) and compute the metrics of Section 5.8 for the system, the baselines, and the ablations. The expert checkpoints that result from the fine-tuning are the only output of this subsystem that the run-time part reads.

6.3.3 Interaction Scenarios

Colorizing an image. Figure 6.5 shows the sequence of calls when a user colorizes an image. The controller calls the subsystems in order, and the experts are called once for each region.

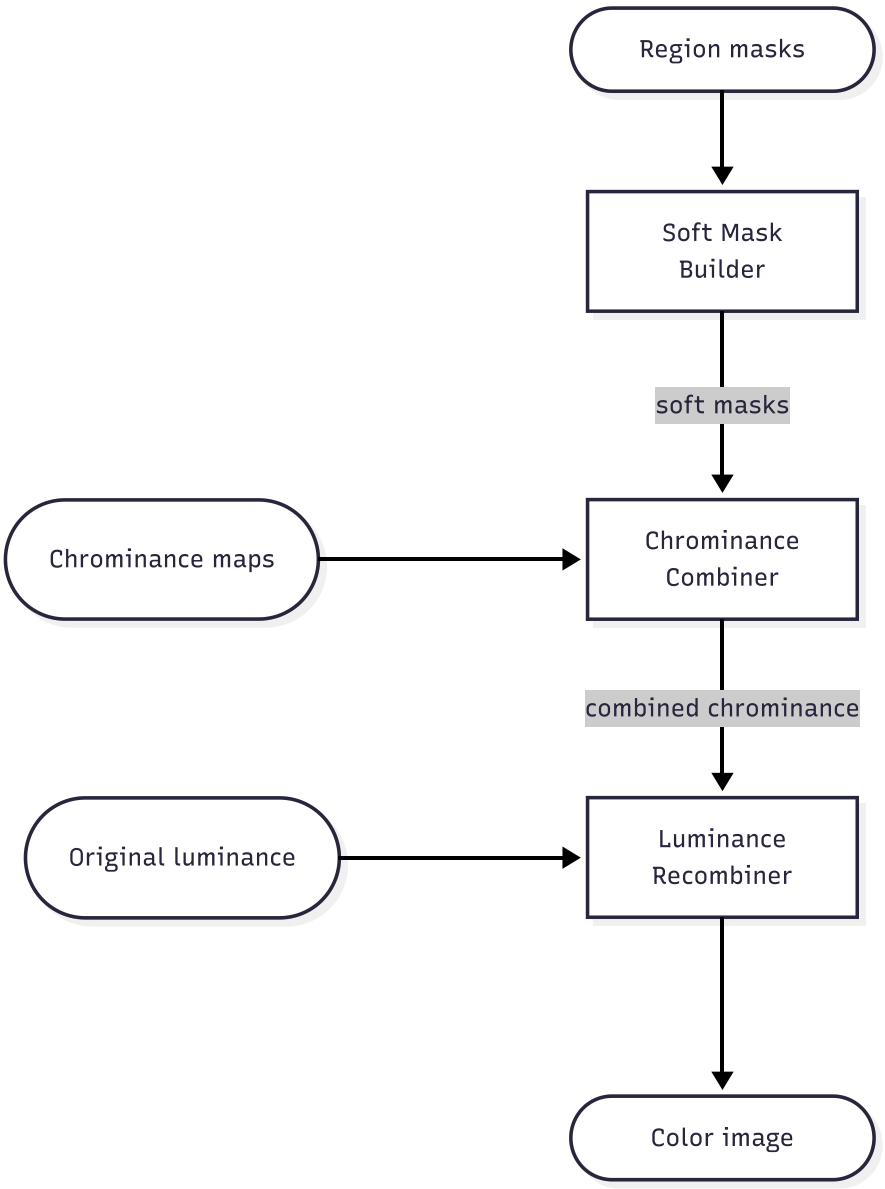


Figure 6.4: Components of the Blending and Reconstruction subsystem.

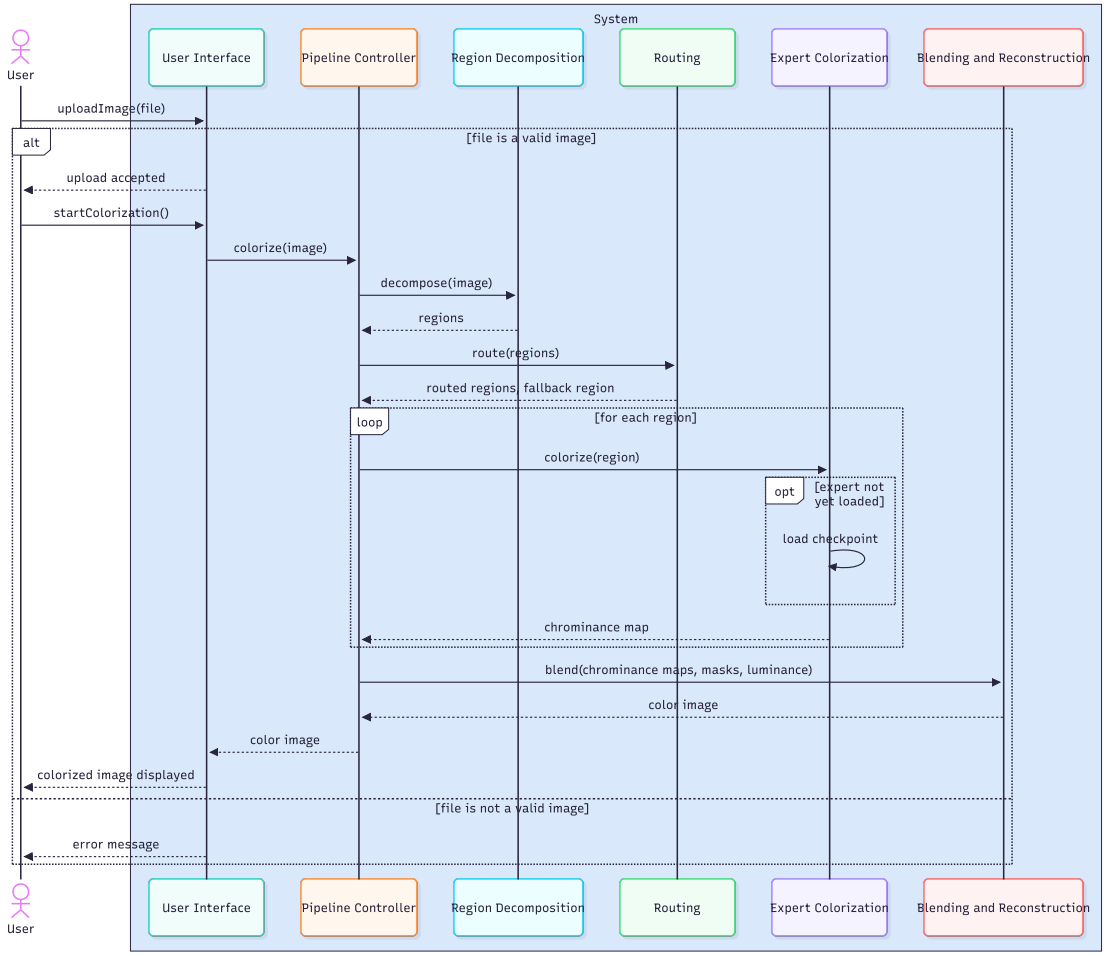


Figure 6.5: System Sequence diagram of colorizing an image.

Adding a new expert. Figure 6.6 shows the steps of Section 5.7 from the point of view of a developer. The existing experts take no part in any of the steps.

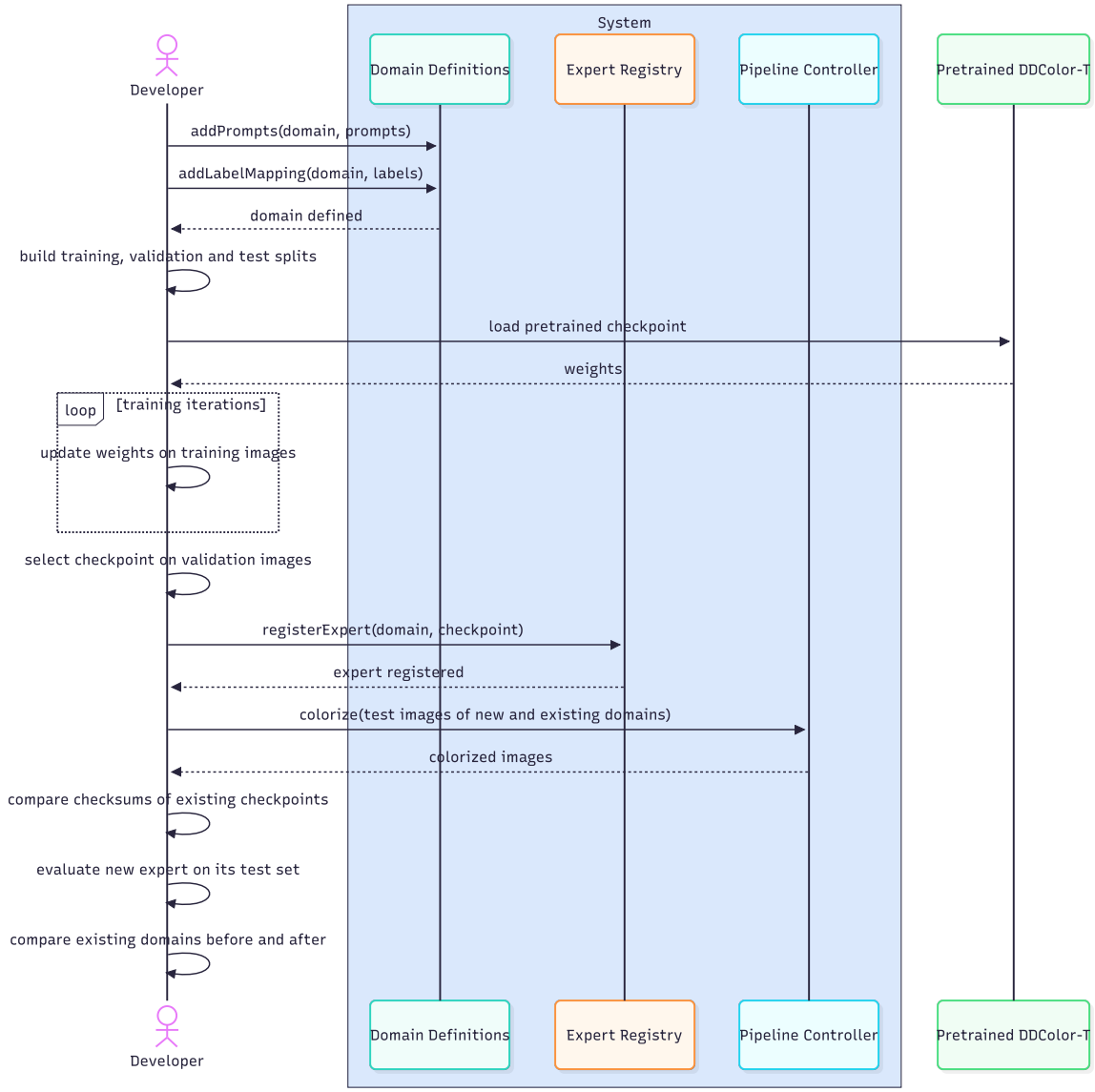


Figure 6.6: Sequence diagram of adding a new expert.

6.3.4 Rationale for the Decomposition

The system is divided along the stages of the method, and each subsystem has one responsibility and a defined input and output. This makes each stage testable on its own, and it means that the segmentation models, the routing rules, or the expert architecture can change without changes to the other subsystems. Three other decompositions were considered.

- **A single script containing the whole method.** This is the quickest to write, but a change to one stage requires changes throughout, and the stages cannot be tested separately. It was rejected.

- **An end-to-end Mixture-of-Experts network with a learned gating router.** This is the usual form of Mixture-of-Experts. It would require training the router together with the experts, and adding an expert would require retraining the router, which conflicts with the extensibility objective. It was rejected.
- **Closed-set semantic segmentation in place of Grounding DINO and the Segment Anything Model.** It would replace two models by one, but the set of regions would be fixed by its label set, so a new domain would require retraining the segmentation model. It was rejected.

6.4 Architectural Strategies

This section describes the main decisions that shaped the architecture, the reasons for them, and the alternatives that were considered. The decisions follow the goals of Section 6.2.

6.4.1 Staged Pipeline with a Central Controller

The system is a sequence of stages, and the Pipeline Controller is the only component that calls them (Section 6.3). Each stage receives its input from the controller and returns its output to it, and no stage calls another. The controller also keeps the data that later stages need but earlier stages do not produce, namely the original image, the original luminance, and the scale factor of the size limit. The alternative is a chain in which each stage calls the next. It would tie every stage to its successor, so replacing or testing one stage would require the others, and the original image would have to be passed through stages that do not use it. A message-based design with queues between the stages was not considered necessary for a prototype that processes one image at a time.

6.4.2 Separation of the User Interface from the Pipeline

The user interface only sends an image to the controller and displays the image it returns. It contains no colorization logic. The reason is that the same pipeline must be run by the evaluation scripts without an interface, and that the interface can be changed without affecting the pipeline. The alternative of placing processing steps in the interface code would make the pipeline impossible to test without the interface.

6.4.3 Reuse of Pretrained Models

Region decomposition and the starting point of every expert rely on pretrained models (Section 6.2). The reason is the compute budget, which does not allow training a detector, a segmentation model, and four colorization networks from scratch. Each pretrained model is used through a thin wrapper that defines the input and output of the stage and hides the library-specific calls. The wrapper limits the effect of a

change of model or library version to one component. The cost of the strategy is a dependence on the availability and licensing of third-party models, which is addressed by recording the versions that are used.

6.4.4 Common Architecture with Independent Weights

All experts use the same network architecture, and each expert has its own weights, trained separately (Section 5.5). Three alternatives were considered. A frozen shared backbone with small per-domain adapters would save storage, but the experts would then share parameters, and a change to the shared part would affect all of them. A single network conditioned on the domain would also share all parameters, and adding a domain would require retraining it. Different architectures for different experts would add code for each one and would mix the effect of the architecture with the effect of the training data in the evaluation. The chosen design keeps each expert replaceable on its own and keeps the comparison between experts and baselines clean.

6.4.5 Registry-Driven Extensibility

The domains known to the system are data and not code. The prompt set, the label-to-domain mapping, and the expert registry are read when the system starts, and the stages use them without reference to specific domains. Adding a domain therefore means adding entries and a checkpoint, and no stage changes (Section 5.7). The alternatives are to write the domains into the code, for example as a chain of conditions, which requires code changes for each new domain, or to use a learned router, which would have to be retrained whenever a domain is added.

6.4.6 Chrominance-Only Prediction with Luminance Preservation

The networks predict only the chrominance channels, and the output uses the luminance of the original image at full resolution (Section 5.6). The alternative of predicting a full RGB image would make the network responsible for structure and brightness that the input already contains, and enlarging its output would blur that detail. With the chosen strategy, the fine detail of the photograph does not pass through the networks, which operate at a reduced size, and only the chrominance is enlarged.

6.4.7 Memory Management

The segmentation models are used for every image and stay loaded. An expert is loaded when a region is first routed to it, and it is released when the image has been processed. The size of the processed image is limited (Section 5.2) so that the memory needed for the activations of the models has an upper bound. The weights of the four experts occupy less than 1 GB in 32-bit precision, so memory is not limiting at

the present number of experts. Loading on demand is chosen so that memory use does not grow with the number of experts, which matters because the system is meant to be extended. The alternative of keeping all experts loaded is simpler and faster at run time, but the memory it requires increases with every added expert. To limit the time lost to loading, the regions of an image are processed in order of their expert, so that each expert is loaded at most once per image.

6.4.8 Error Detection and Recovery

The system either returns a complete colorized image or reports an error. It never returns a partly colorized image. The following situations are handled.

- **Invalid input.** A file that is not a valid image is rejected before the pipeline starts, and the user receives an error message.
- **No region detected, or a label that cannot be routed.** The pixels concerned are uncovered, and the fallback expert colorizes them (Section 5.3.3). If no region is found, the whole image is colorized by the fallback expert.
- **Empty region after overlap resolution.** A region whose mask has no remaining pixels is dropped (Section 5.3.2).
- **Missing or unreadable expert checkpoint.** The regions of that expert are given to the fallback expert, and a warning is recorded.
- **Insufficient memory.** The request fails with an error message and no partial result is returned.

6.4.9 Control Flow and Concurrency

The stages run one after the other, since each needs the output of the previous one. Inside the expert stage, regions are processed one at a time. Regions routed to the same expert all have the same input size, so they could be processed together in a single batch, which is a possible optimization that the prototype does not use. Requests are handled one at a time, because the system runs on a single GPU and serves one user at a time. Handling several requests at once would add scheduling and memory management that a prototype does not need.

6.5 Domain Model and Class Diagram

Figure 6.7 shows the classes. The upper part contains the interface and the controller. Below them, the classes are arranged in four columns, one for each subsystem of Section 6.3: region decomposition, routing, extraction and expert colorization, and blending. The first class of each column is the class that

the controller calls, and the classes below it are its components. The data classes that the subsystems exchange are shown at the bottom.

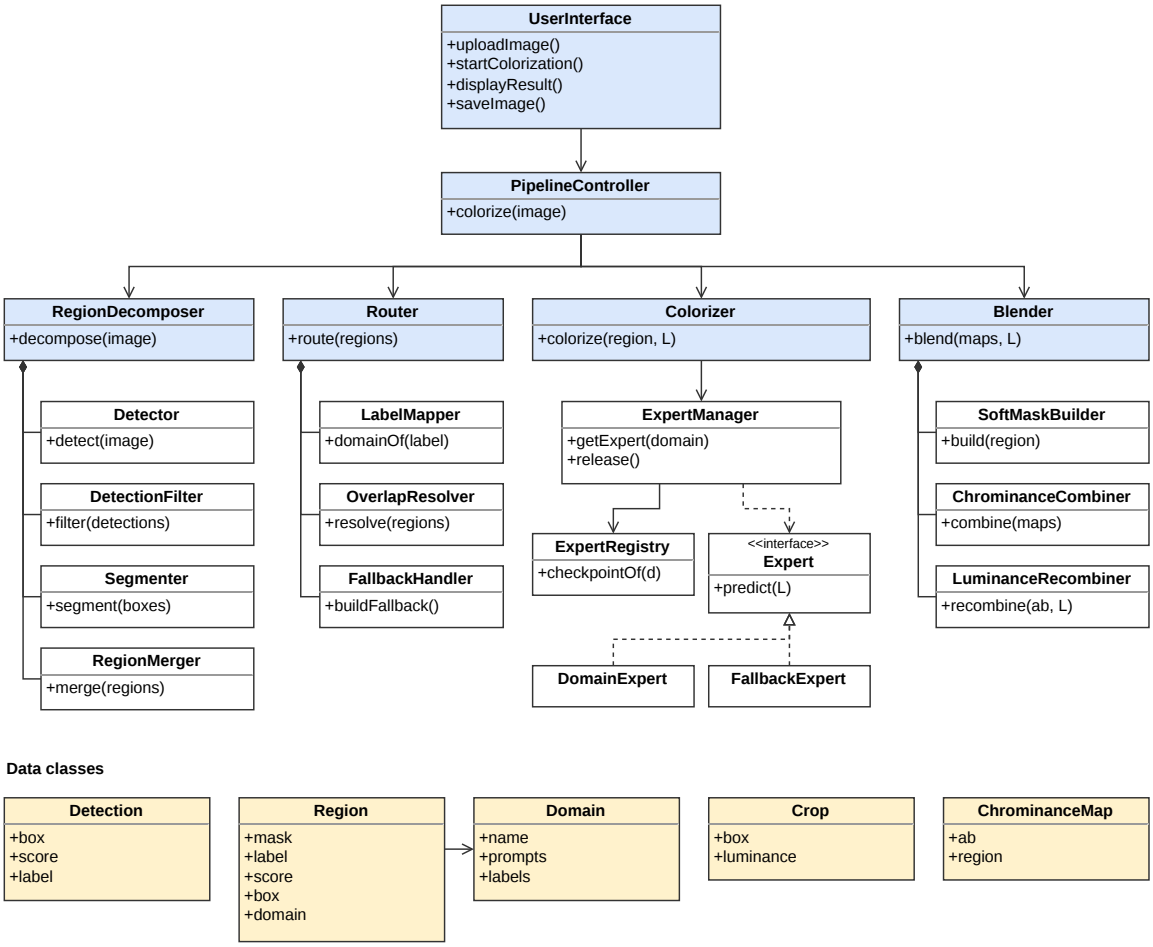


Figure 6.7: Class diagram of the run-time pipeline.

6.6 Policies and Tactics

6.6.1 Tools and Libraries

The pipeline is written in Python. The networks run on PyTorch, and the pretrained models are used through the libraries released with them. The web interface is built with Next.js for the frontend and FastAPI for the backend that exposes the pipeline to it. Library and model versions are fixed, because the released code of the pretrained models depends on specific versions.

6.6.2 Coding Conventions

The code follows the PEP 8 style guide. Each class of Figure 6.7 is placed in its own module, and each class and public method has a short docstring that states its input and output. Each stage is called only through the interface of its entry class, so that no module depends on the internals of another.

6.6.3 Testing

Each stage is tested alone with small fixed inputs: for example the filter on a list of detections with known scores, the overlap resolver on two overlapping masks, and the blender on two regions with known chrominance. The complete pipeline is then run on a few test images and the intermediate results are inspected visually. The quality of the system is measured with the evaluation of Section 5.8, and not by these tests.

6.6.4 Reproducibility and Maintenance

Library versions, model checkpoint names, and random seeds are recorded with each experiment (Section 6.2). Changes to the prompt set, the label mapping, or the list of experts are made as data changes and do not modify the code of the stages (Section 6.4).

6.6.5 Traceability

Table 6.3 relates each project objective to the components that implement it and to the part of the evaluation that tests it.

Table 6.3: Relation between objectives, components, and evaluation.

Objective	Components	Evaluation
Region decomposition	RegionDecomposer and its components	mIoU (Section 5.8)
Expert colorization quality	RegionExtractor, ExpertManager, experts	FID, LPIPS, PSNR, SSIM, ΔE
Routing	Router, LabelMapper, OverlapResolver	Routing accuracy
Blending	Blender and its components	Boundary ΔE, FID, LPIPS
Extensibility	ExpertRegistry, Domain	Extensibility test

Conclusion

This chapter presented the design of the colorization system. The system is a pipeline of subsystems that are called in order by a controller: region decomposition, routing, expert colorization, and blending. A basic web interface sits on top of it. The design follows the goals of modularity, extensibility, and

preservation of the input, and it works within the limits of a single GPU of moderate memory. The class diagram defines the classes that implement the subsystems, and the policies describe how they are built and tested. The next chapter describes the implementation and reports the results of the evaluation.

Bibliography

[1] X. Cong, Y. Wu, Q. Chen, and C. Lei, "Automatic controllable colorization via imagination," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2609–2619, 2024.

[2] T. Ren, S. Liu, A. Zeng, J. Lin, K. Li, H. Cao, J. Chen, X. Huang, Y. Chen, F. Yan, Z. Zeng, H. Zhang, F. Li, J. Yang, H. Li, Q. Jiang, and L. Zhang, "Grounded SAM: Assembling open-world models for diverse visual tasks," *arXiv preprint arXiv:2401.14159*, 2024.

[3] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024.

[4] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, "Segment anything," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, pp. 4015–4026, 2023.

[5] H. Caesar, J. Uijlings, and V. Ferrari, "COCO-Stuff: Thing and stuff classes in context," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 1209–1218, 2018.

[6] R. Zhang, P. Isola, and A. A. Efros, "Colorful image colorization," in *Computer Vision – ECCV 2016*, vol. 9907 of *Lecture Notes in Computer Science*, pp. 649–666, 2016.

[7] S. Iizuka, E. Simo-Serra, and H. Ishikawa, "Let there be color!: Joint end-to-end learning of global and local image priors for automatic image colorization with simultaneous classification," *ACM Transactions on Graphics*, vol. 35, no. 4, pp. 110:1–110:11, 2016.

[8] R. Zhang, J.-Y. Zhu, P. Isola, X. Geng, A. S. Lin, T. Yu, and A. A. Efros, "Real-time user-guided image colorization with learned deep priors," *ACM Transactions on Graphics*, vol. 36, no. 4, 2017.

[9] M. He, D. Chen, J. Liao, P. V. Sander, and L. Yuan, "Deep exemplar-based colorization," *ACM Transactions on Graphics*, vol. 37, no. 4, pp. 47:1–47:16, 2018.

[10] M. Kumar, D. Weissenborn, and N. Kalchbrenner, "Colorization transformer," in *Proceedings of the 9th International Conference on Learning Representations (ICLR)*, 2021.

[11] Y. Wu, X. Wang, Y. Li, H. Zhang, X. Zhao, and Y. Shan, "Towards vivid and diverse image colorization with generative color prior," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 14377–14386, 2021.

[12] G. Kim, K. Kang, S. Kim, H. Lee, S. Kim, J. Kim, S.-H. Baek, and S. Cho, "BigColor: Colorization using a generative color prior for natural images," in *Computer Vision – ECCV 2022*, Lecture Notes in Computer Science, pp. 350–366, 2022.

[13] C. Saharia, W. Chan, H. Chang, C. A. Lee, J. Ho, T. Salimans, D. J. Fleet, and M. Norouzi, "Palette: Image-to-image diffusion models," in *ACM SIGGRAPH 2022 Conference Proceedings*, 2022.

[14] X. Kang, T. Yang, W. Ouyang, P. Ren, L. Li, and X. Xie, "DDColor: Towards photo-realistic image colorization via dual decoders," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 328–338, 2023.

[15] X. Kang, T. Yang, W. Ouyang, P. Ren, L. Li, and X. Xie, "DDColor: Official pytorch implementation." GitHub repository, 2023. Accessed: 2026-10-03.

[16] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. V. Le, G. E. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.

[17] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[18] X. O. He, "Mixture of a million experts," *arXiv preprint arXiv:2407.04153*, 2024.

[19] H. Zhou, R. Yang, L. Tang, G. Qin, Y. Zhang, R. Hu, and X. Li, "Gamma: Toward generic image assessment with mixture of assessment experts," in *Proceedings of the 33rd ACM International Conference on Multimedia*, pp. 8815–8824, 2025.

[20] C. Fan *et al.*, "RS-MoEGAN: Sparse mixture-of-experts for cross-modal remote sensing image generation," *Chinese Journal of Aeronautics*, p. 104420, 2026.

[21] J.-W. Su, H.-K. Chu, and J.-B. Huang, "Instance-aware image colorization," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7965–7974, 2020.

[22] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick,

"Microsoft COCO: Common objects in context," in *Computer Vision – ECCV 2014*, Lecture Notes in Computer Science, pp. 740–755, 2014.

[23] B. Zhou, H. Zhao, X. Puig, S. Fidler, A. Barriuso, and A. Torralba, "Scene parsing through ADE20K dataset," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 633–641, 2017.

[24] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, "The cityscapes dataset for semantic urban scene understanding," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3213–3223, 2016.

[25] S. Anwar, M. Tahir, C. Li, A. Mian, F. S. Khan, and A. W. Muzaffar, "Image colorization: A survey and dataset," *Information Fusion*, vol. 114, p. 102720, 2025.

[26] G. Larsson, M. Maire, and G. Shakhnarovich, "Learning representations for automatic colorization," in *Computer Vision – ECCV 2016*, Lecture Notes in Computer Science, pp. 577–593, 2016.

[27] R. Zhang, P. Isola, and A. A. Efros, "Colorful image colorization: Official implementation." GitHub repository, 2016. Accessed: 2026-10-03.

[28] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 6626–6637, 2017.

[29] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 586–595, 2018.

[30] D. Hasler and S. Süsstrunk, "Measuring colourfulness in natural images," in *Proc. IS&T/SPIE Electronic Imaging 2003: Human Vision and Electronic Imaging VIII*, vol. 5007, pp. 87–95, 2003.